Kubernetes PHP Client (k8s-client-php) — Developer Book

A comprehensive, chapter-wise guide to interacting with the Kubernetes API using PHP — from first connection to production-grade cluster management, event watching, RBAC, and real-world automation.

Table of Contents

#	Chapter	Topics Covered
01	**Introduction**	What is k8s-client-php, when to use it, architecture overview, PHP vs kubectl
02	**Installation & Setup**	Composer install, authentication methods, KubeConfig, in-cluster config
03	**Connecting to a Cluster**	Local cluster, remote cluster, service accounts, TLS, context switching
04	**Working with Pods**	List, get, create, delete pods, exec into pods, logs, pod status
05	**Deployments & ReplicaSets**	Create/update deployments, rolling updates, scale, rollback
06	**Services & Networking**	ClusterIP, NodePort, LoadBalancer services, endpoints
07	**ConfigMaps & Secrets**	Create, read, update ConfigMaps and Secrets via PHP
08	**Persistent Volumes & Storage**	PV, PVC, StorageClass management through the API
09	**Namespaces & RBAC**	Create namespaces, Roles, ClusterRoles, RoleBindings, ServiceAccounts
10	**Ingress**	Create and manage Ingress rules, TLS, annotations
11	**StatefulSets & DaemonSets**	Managing stateful workloads and node-level agents

#	Chapter	Topics Covered
12	**Jobs & CronJobs**	One-off tasks and scheduled jobs via PHP
13	**Watching Events & Real-Time Updates**	Watch API, streaming events, detecting pod restarts
14	**Custom Resources (CRDs)**	Listing and managing CRDs, dynamic client usage
15	**Error Handling & Retry Logic**	API exceptions, status codes, retries, rate limiting
16	**Production Patterns**	Operator pattern, leader election, reconciliation loops, multi-cluster

---

How to Read This Book

If you're brand new to k8s-client-php: Read 01 → 02 → 03 → 04 in order. This gives you the core mental model.

If you're building a deployment tool: Read 05 → 06 → 07 → 10.

If you're managing cluster resources programmatically: Read 09 → 08 → 12 → 13.

If you're writing a Kubernetes Operator in PHP: Read 13 → 14 → 16.

If something broke: Jump to Chapter 15 (Error Handling & Retry Logic).

Quick Reference

```php
use Maclof\Kubernetes\Client;

$client = new Client([
    'master'      => 'https://k8s.example.com:6443',
    'ca_cert'     => '/path/to/ca.crt',
    'client_cert' => '/path/to/client.crt',
    'client_key'  => '/path/to/client.key',
]);
```

```php
// List all pods in default namespace
$pods = $client->pods()->find();

// Get a specific pod
$pod = $client->pods()->findByName('my-pod');

// Delete a pod
$client->pods()->deleteByName('my-pod');
```

Key Libraries

Library	GitHub	Notes
`maclof/kubernetes-client`	maclof/kubernetes-client	Most widely used PHP K8s client
`php-k8s/php-k8s`	renoki-co/php-k8s	Laravel-friendly, fluent API
Raw HTTP via Guzzle	—	For advanced/custom resources

Chapter 1: Introduction to k8s-client-php

What is k8s-client-php?

k8s-client-php refers to PHP libraries that wrap the Kubernetes REST API, letting you manage cluster resources (Pods, Deployments, Services, etc.) directly from PHP code — without shelling out to `kubectl`.

The two most popular options are:

Library	Style	Best For
`maclof/kubernetes-client`	Procedural, fluent	General PHP apps
`renoki-co/php-k8s`	Eloquent-like, Laravel	Laravel/Lumen projects

This book uses `maclof/kubernetes-client` as the primary example, with `php-k8s` patterns shown where relevant.

Why Use a PHP Client Instead of kubectl?

Scenario	Use kubectl	Use PHP Client
One-off cluster debugging	Yes	No
CI/CD pipeline shell script	Yes	Maybe
Web dashboard managing resources	No	Yes
SaaS platform provisioning namespaces per customer	No	Yes
Kubernetes Operator written in PHP	No	Yes
Automated reconciliation loop	No	Yes

> **Rule:** If a human is running the command → use `kubectl`. If your application is doing it programmatically → use a PHP client.

How the Kubernetes API Works

The Kubernetes API is a **RESTful HTTP API** with a well-defined schema:

```
GET    /api/v1/namespaces/{namespace}/pods          → list pods
GET    /api/v1/namespaces/{namespace}/pods/{name}   → get pod
POST   /api/v1/namespaces/{namespace}/pods          → create pod
PATCH  /api/v1/namespaces/{namespace}/pods/{name}   → update pod
DELETE /api/v1/namespaces/{namespace}/pods/{name}   → delete pod
```

All responses are JSON. The PHP client libraries are thin wrappers that:

1. Build the correct URL and HTTP method
2. Serialize PHP arrays/objects to JSON
3. Handle authentication headers
4. Parse responses back into PHP objects

API Groups and Versions

Kubernetes has multiple API groups:

API Group	Path	Contains
Core	`/api/v1`	Pods, Services, ConfigMaps, Secrets, PVs
Apps	`/apis/apps/v1`	Deployments, StatefulSets, DaemonSets, ReplicaSets
Batch	`/apis/batch/v1`	Jobs, CronJobs
Networking	`/apis/networking.k8s.io/v1`	Ingress, NetworkPolicy
RBAC	`/apis/rbac.authorization.k8s.io/v1`	Roles, ClusterRoles, Bindings
Custom	`/apis/{group}/{version}`	CRDs

The PHP client handles routing to the correct API group automatically.

Architecture Overview

```
Your PHP App
     │
     │  HTTP/TLS
     ▼
Kubernetes API Server  (port 6443)
     │
     ├── etcd          (stores all state)
     ├── Scheduler
     └── Controllers
```

Your PHP code:

1. Authenticates with the API Server (cert, token, or kubeconfig)
2. Sends REST requests to create/read/update/delete resources
3. The API Server validates and stores the changes

4. Controllers reconcile toward the new desired state

What You Can Do with the PHP Client

- **Read resources:** List pods, get deployment status, check service endpoints
- **Write resources:** Create namespaces, deploy applications, create secrets
- **Watch resources:** Stream real-time events (pod restarts, deployment rollouts)
- **Execute into pods:** Run commands inside running containers
- **Manage RBAC:** Create roles, bindings, service accounts programmatically
- **Handle CRDs:** Interact with custom resource types

What You Cannot Do (Limitations)

- Cannot replace the full power of `kubectl` for interactive debugging
- No built-in leader election (you implement it yourself)
- CRD support varies by library — raw HTTP is sometimes needed
- Watching streams (long-polling) requires careful connection management in PHP-FPM environments

---

PHP vs Other Language Clients

Feature	PHP Client	Go client-go	Python kubernetes
Official support	Community	Official (CNCF)	Official (CNCF)
Operator frameworks	None	controller-runtime	kopf
Stability	Good	Excellent	Excellent
Ecosystem	Laravel-friendly	Best for operators	Strong ML/data
Async support	Limited	Yes (goroutines)	Yes (asyncio)

> For building production operators or controllers, Go is the standard. PHP clients are ideal for web dashboards, SaaS provisioning, and CI/CD integrations within existing PHP stacks.

Chapter 2: Installation & Setup

Requirements

- PHP >= 7.4 (8.x recommended)
- Composer
- A running Kubernetes cluster (Minikube, k3s, kind, or cloud-managed)
- kubectl configured (for getting credentials)

Installing maclof/kubernetes-client

```
composer require maclof/kubernetes-client


# Verify installation
composer show maclof/kubernetes-client
```

Key dependencies it installs:

- `guzzlehttp/guzzle` — HTTP client
- `symfony/yaml` — YAML parsing

Installing renoki-co/php-k8s (Laravel alternative)

```
composer require renoki-co/php-k8s
```

For Laravel service provider:

```
php artisan vendor:publish --provider="RenokiCo\PhpK8s\PhpK8sServiceProvider"
```

Authentication Methods

Method 1: kubeconfig File (Local Development)

The easiest method — reads the same config `kubectl` uses.

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig(
    kubeConfigPath: '/home/user/.kube/config',
    context: 'my-cluster-context'   // optional, uses current-context if
omitted
);
```

Get your kubeconfig context names:

```
kubectl config get-contexts
kubectl config current-context
```

Method 2: Service Account Token (In-Cluster / Production)

When your PHP app runs **inside** the cluster (as a Pod), Kubernetes mounts credentials automatically:

```
/var/run/secrets/kubernetes.io/serviceaccount/token    ← Bearer token
/var/run/secrets/kubernetes.io/serviceaccount/ca.crt   ← CA certificate
```

```php
use Maclof\Kubernetes\Client;

$client = new Client([
    'master'          => 'https://kubernetes.default.svc',
    'ca_cert'         =>
'/var/run/secrets/kubernetes.io/serviceaccount/ca.crt',
    'token'           =>
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/token'),
    'namespace'       =>
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/namespace'),
]);
```

This is the **recommended approach for production** — no credentials are baked into your image.

Method 3: Client Certificate + Key

```php
$client = new Client([
    'master'      => 'https://k8s.example.com:6443',
    'ca_cert'     => '/path/to/ca.crt',
    'client_cert' => '/path/to/client.crt',
    'client_key'  => '/path/to/client.key',
]);
```

Extract certs from kubeconfig:

```
# Get CA cert
kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' | base64 -d > ca.crt

# Get client cert
kubectl config view --raw -o jsonpath='{.users[0].user.client-certificate-data}' | base64 -d > client.crt

# Get client key
kubectl config view --raw -o jsonpath='{.users[0].user.client-key-data}' | base64 -d > client.key
```

Method 4: Bearer Token

```php
$client = new Client([
    'master' => 'https://k8s.example.com:6443',
    'ca_cert' => '/path/to/ca.crt',
    'token'  => 'eyJhbGci...your-token...',
]);
```

Get a ServiceAccount token:

```
# Kubernetes 1.24+
kubectl create token my-service-account -n default
```

```
# Or create a long-lived secret (older approach)
kubectl apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: my-sa-token
  annotations:
    kubernetes.io/service-account.name: my-service-account
type: kubernetes.io/service-account-token
EOF

kubectl get secret my-sa-token -o jsonpath='{.data.token}' | base64 -d
```

Environment-Based Configuration

Keep credentials out of code — load from environment:

```php
$client = new Client([
    'master'  => getenv('K8S_MASTER'),
    'ca_cert' => getenv('K8S_CA_CERT_PATH'),
    'token'   => getenv('K8S_TOKEN'),
]);
```

.env file (development only — never commit):

```
K8S_MASTER=https://127.0.0.1:6443
K8S_CA_CERT_PATH=/home/user/.minikube/ca.crt
K8S_TOKEN=eyJhbGci...
```

Namespace Configuration

By default, the client targets the `default` namespace. You can set it globally or per-request:

```php
// Global namespace
$client = new Client([
    'master'    => '...',
    'namespace' => 'production',
```

```
    // ...
]);

// Switch namespace at runtime
$client->setNamespace('staging');
$pods = $client->pods()->find();

$client->setNamespace('production');
$deployments = $client->deployments()->find();
```

Verifying the Connection

php

```
<?php
require 'vendor/autoload.php';

use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

try {
    $namespaces = $client->namespaces()->find();
    echo "Connected! Namespaces found: " . count($namespaces) . "\n";
    foreach ($namespaces as $ns) {
        echo " - " . $ns->getMetadata('name') . "\n";
    }
} catch (\Exception $e) {
    echo "Connection failed: " . $e->getMessage() . "\n";
}
```

Minikube Quick Start

```
# Start minikube
minikube start

# Get API endpoint and credentials
minikube ip                    # e.g. 192.168.49.2
kubectl cluster-info           # Shows API server URL
```

```
# Copy minikube certs
cp ~/.minikube/ca.crt /tmp/k8s-ca.crt
cp ~/.minikube/profiles/minikube/client.crt /tmp/k8s-client.crt
cp ~/.minikube/profiles/minikube/client.key /tmp/k8s-client.key
```

```php
$client = new Client([
    'master'      => 'https://192.168.49.2:8443',
    'ca_cert'     => '/tmp/k8s-ca.crt',
    'client_cert' => '/tmp/k8s-client.crt',
    'client_key'  => '/tmp/k8s-client.key',
]);
```

Client Configuration Options Reference

Option	Type	Description
`master`	string	API server URL
`ca_cert`	string	Path to CA certificate file
`client_cert`	string	Path to client certificate
`client_key`	string	Path to client private key
`token`	string	Bearer token string
`namespace`	string	Default namespace (default: `default`)
`timeout`	int	HTTP request timeout in seconds
`verify`	bool	Verify TLS (disable only for dev)

Chapter 3: Connecting to a Cluster

Connection Architecture

```
Your PHP App
     |
```

```
      | HTTPS (port 6443)
      ▼
┌──────────────────────────┐
│   Kubernetes API Server  │
│                          │
│  Authenticates via:      │
│  - Client certificate    │
│  - Bearer token          │
│  - Basic auth (legacy)   │
└──────────────────────────┘
```

Local Cluster (Minikube / kind / k3s)

Minikube

```php
use Maclof\Kubernetes\Client;

// Option A: Auto-detect from kubeconfig
$client = Client::fromKubeConfig();

// Option B: Explicit minikube certs
$client = new Client([
    'master'      => shell_exec('minikube ip') . ':8443',
    'ca_cert'     => $_SERVER['HOME'] . '/.minikube/ca.crt',
    'client_cert' => $_SERVER['HOME'] .
'/.minikube/profiles/minikube/client.crt',
    'client_key'  => $_SERVER['HOME'] .
'/.minikube/profiles/minikube/client.key',
]);
```

kind (Kubernetes in Docker)

```
# Create cluster
kind create cluster --name dev

# Get config
kind get kubeconfig --name dev > /tmp/kind-config.yaml
```

```php
$client = Client::fromKubeConfig(
    kubeConfigPath: '/tmp/kind-config.yaml'
);
```

k3s

```
# k3s writes kubeconfig here
sudo cat /etc/rancher/k3s/k3s.yaml
# Copy to user path
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER ~/.kube/config
```

```php
$client = Client::fromKubeConfig();  // reads ~/.kube/config automatically
```

Remote / Cloud Cluster

GKE (Google Kubernetes Engine)

```
# Authenticate and get credentials
gcloud container clusters get-credentials my-cluster --region us-central1

# kubeconfig is updated automatically
kubectl config current-context   # gke_project_region_cluster
```

```php
$client = Client::fromKubeConfig(
    context: 'gke_my-project_us-central1_my-cluster'
);
```

EKS (AWS Elastic Kubernetes Service)

```
aws eks update-kubeconfig --region us-east-1 --name my-cluster
```

```php
// EKS uses AWS IAM token — use kubectl proxy for PHP simplicity
// OR use aws-iam-authenticator token

$token = shell_exec('aws eks get-token --cluster-name my-cluster --query
```

```
  "status.token" -o text');

  $client = new Client([
      'master'  => 'https://ABCDEF.gr7.us-east-1.eks.amazonaws.com',
      'ca_cert' => '/path/to/eks-ca.crt',
      'token'   => trim($token),
  ]);
```

AKS (Azure Kubernetes Service)

```
  az aks get-credentials --resource-group myRG --name my-cluster
```

```php
  $client = Client::fromKubeConfig(
      context: 'my-cluster'
  );
```

In-Cluster Connection (Pod Running Inside K8s)

When your PHP app runs as a Pod inside Kubernetes, Kubernetes automatically injects credentials:

```yaml
  # Your Pod automatically gets:
  # /var/run/secrets/kubernetes.io/serviceaccount/token
  # /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
  # /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

```php
  function createInClusterClient(): Client
  {
      $tokenPath = '/var/run/secrets/kubernetes.io/serviceaccount/token';
      $caPath    = '/var/run/secrets/kubernetes.io/serviceaccount/ca.crt';
      $nsPath    = '/var/run/secrets/kubernetes.io/serviceaccount/namespace';

      return new Client([
          'master'    => 'https://kubernetes.default.svc',
          'ca_cert'   => $caPath,
          'token'     => file_get_contents($tokenPath),
          'namespace' => file_get_contents($nsPath),
      ]);
  }
```

Context Switching

Work with multiple clusters or namespaces in one PHP script:

```php
use Maclof\Kubernetes\Client;

// Load two different contexts
$dev  = Client::fromKubeConfig(context: 'dev-cluster');
$prod = Client::fromKubeConfig(context: 'prod-cluster');

// Query both
$devPods  = $dev->pods()->find();
$prodPods = $prod->pods()->find();

echo "Dev pods:  " . count($devPods)  . "\n";
echo "Prod pods: " . count($prodPods) . "\n";
```

Namespace Switching at Runtime

```php
$client = Client::fromKubeConfig();

$namespaces = ['default', 'kube-system', 'production', 'staging'];

foreach ($namespaces as $ns) {
    $client->setNamespace($ns);
    $pods = $client->pods()->find();
    echo "[$ns] Pods: " . count($pods) . "\n";
}
```

Using kubectl proxy (Simplest Dev Setup)

Avoid certificate complexity by proxying through kubectl:

```
# Start kubectl proxy (runs on localhost:8001)
kubectl proxy &
```

```php
// No auth needed — kubectl proxy handles it
$client = new Client([
    'master'    => 'http://127.0.0.1:8001',
    'verify'    => false,
]);

$pods = $client->pods()->find();
```

> Only use this for local development. Never expose `kubectl proxy` in production.

Disabling TLS Verification (Dev Only)

```php
$client = new Client([
    'master' => 'https://...',
    'verify' => false,   // NEVER use in production
    'token'  => '...',
]);
```

Connection Troubleshooting

Error	Likely Cause	Fix
`Connection refused`	Wrong port or API server down	Check `kubectl cluster-info`
`SSL certificate error`	Wrong CA cert	Verify `ca_cert` path
`401 Unauthorized`	Invalid/expired token	Refresh token
`403 Forbidden`	Insufficient RBAC permissions	Grant role to ServiceAccount
`Timeout`	Network issue / firewall	Check cluster network access

```php
// Test connection with verbose error reporting
try {
    $version = $client->getVersion();
    echo "K8s version: " . $version['gitVersion'] . "\n";
} catch (\GuzzleHttp\Exception\ConnectException $e) {
    echo "Cannot connect to cluster: " . $e->getMessage() . "\n";
} catch (\GuzzleHttp\Exception\ClientException $e) {
    echo "HTTP error " . $e->getResponse()->getStatusCode() . ": "
        . $e->getResponse()->getBody() . "\n";
}
```

Chapter 4: Working with Pods

Pod Resource Methods

Method	Description
`$client->pods()->find()`	List all pods in current namespace
`$client->pods()->findByName('name')`	Get a single pod
`$client->pods()->create($model)`	Create a new pod
`$client->pods()->delete($model)`	Delete a pod
`$client->pods()->deleteByName('name')`	Delete pod by name
`$client->pods()->find(['label' => 'value'])`	Filter pods by label
`$client->pods()->logs('name')`	Fetch pod logs

Listing Pods

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

// List all pods in current namespace
$pods = $client->pods()->find();
```

```
foreach ($pods as $pod) {
    $name      = $pod->getMetadata('name');
    $namespace = $pod->getMetadata('namespace');
    $phase     = $pod->getStatus()['phase'] ?? 'Unknown';
    $nodeName  = $pod->getSpec()['nodeName'] ?? 'unscheduled';

    printf("%-40s %-15s %-10s %s\n", $name, $namespace, $phase, $nodeName);
}
```

List Pods Across All Namespaces

```php
$client->setNamespace('');  // empty string = all namespaces

$allPods = $client->pods()->find();
echo "Total pods cluster-wide: " . count($allPods) . "\n";
```

Filtering Pods by Label

```php
// Pods with label app=nginx
$nginxPods = $client->pods()->setLabelSelector([
    'app' => 'nginx',
])->find();

// Multiple labels (AND condition)
$prodNginx = $client->pods()->setLabelSelector([
    'app' => 'nginx',
    'env' => 'production',
])->find();

// Field selector
$runningPods = $client->pods()->setFieldSelector([
    'status.phase' => 'Running',
])->find();
```

Getting a Single Pod

```php
$pod = $client->pods()->findByName('my-nginx-pod');

if ($pod === null) {
    echo "Pod not found\n";
    exit(1);
}

// Access metadata
echo "Name:      " . $pod->getMetadata('name') . "\n";
echo "Namespace: " . $pod->getMetadata('namespace') . "\n";
echo "Node:      " . ($pod->getSpec()['nodeName'] ?? 'N/A') . "\n";
echo "Phase:     " . ($pod->getStatus()['phase'] ?? 'N/A') . "\n";

// Container statuses
$containerStatuses = $pod->getStatus()['containerStatuses'] ?? [];
foreach ($containerStatuses as $cs) {
    echo "Container: {$cs['name']}  Ready: " . ($cs['ready'] ? 'yes' : 'no') .
"\n";
}
```

Creating a Pod

```php
use Maclof\Kubernetes\Models\Pod;

$pod = new Pod([
    'metadata' => [
        'name'      => 'my-nginx',
        'namespace' => 'default',
        'labels'    => [
            'app' => 'nginx',
            'env' => 'dev',
        ],
    ],
    'spec' => [
        'containers' => [
            [
                'name'  => 'nginx',
                'image' => 'nginx:1.25-alpine',
                'ports' => [
                    ['containerPort' => 80],
                ],
```

```
                'resources' => [
                    'requests' => ['memory' => '64Mi', 'cpu' => '100m'],
                    'limits'   => ['memory' => '128Mi', 'cpu' => '200m'],
                ],
                'env' => [
                    ['name' => 'ENV',  'value' => 'production'],
                    ['name' => 'PORT', 'value' => '8080'],
                ],
            ],
        ],
        'restartPolicy' => 'Always',
    ],
]);

$created = $client->pods()->create($pod);
echo "Pod created: " . $created->getMetadata('name') . "\n";
```

Creating a Pod with Multiple Containers

php

```
$pod = new Pod([
    'metadata' => ['name' => 'sidecar-example'],
    'spec' => [
        'containers' => [
            [
                'name'  => 'app',
                'image' => 'myapp:1.0',
                'ports' => [['containerPort' => 8080]],
            ],
            [
                'name'  => 'log-shipper',
                'image' => 'fluent/fluent-bit:latest',
                'volumeMounts' => [
                    ['name' => 'app-logs', 'mountPath' => '/var/log/app'],
                ],
            ],
        ],
        'volumes' => [
            ['name' => 'app-logs', 'emptyDir' => []],
        ],
    ],
]);
```

```
$client->pods()->create($pod);
```

Pod Logs

```php
// Get current logs
$logs = $client->pods()->logs('my-nginx');
echo $logs;

// Get logs for a specific container (multi-container pod)
$logs = $client->pods()->logs('sidecar-pod', ['container' => 'log-shipper']);

// Last N lines
$logs = $client->pods()->logs('my-nginx', ['tailLines' => 100]);

// Logs since a duration
$logs = $client->pods()->logs('my-nginx', ['sinceSeconds' => 3600]);

// Previous container instance (for CrashLoopBackOff debugging)
$logs = $client->pods()->logs('my-nginx', ['previous' => true]);
```

Checking Pod Status

```php
function getPodStatus(Client $client, string $podName): string
{
    $pod = $client->pods()->findByName($podName);

    if ($pod === null) {
        return 'NOT_FOUND';
    }

    $phase = $pod->getStatus()['phase'] ?? 'Unknown';

    $containerStatuses = $pod->getStatus()['containerStatuses'] ?? [];
    foreach ($containerStatuses as $cs) {
        if (isset($cs['state']['waiting']['reason'])) {
            return $cs['state']['waiting']['reason'];  // CrashLoopBackOff,
```

```
    ImagePullBackOff, etc.
            }
        }

        return $phase;
    }

    // Wait for a pod to be Running
    function waitForPodRunning(Client $client, string $podName, int $timeoutSec =
    60): bool
    {
        $start = time();
        while (time() - $start < $timeoutSec) {
            $status = getPodStatus($client, $podName);
            if ($status === 'Running') {
                return true;
            }
            echo "Pod status: $status — waiting...\n";
            sleep(2);
        }
        return false;
    }
```

Deleting a Pod

php

```
    // Delete by name
    $client->pods()->deleteByName('my-nginx');
    echo "Pod deleted\n";

    // Delete with grace period (seconds)
    $pod = $client->pods()->findByName('my-nginx');
    $client->pods()->delete($pod, [
        'gracePeriodSeconds' => 0,  // force delete
    ]);

    // Delete all pods with a label
    $oldPods = $client->pods()->setLabelSelector(['app' => 'old-app'])->find();
    foreach ($oldPods as $pod) {
        $client->pods()->delete($pod);
        echo "Deleted: " . $pod->getMetadata('name') . "\n";
    }
```

Pod IP and Networking Info

```php
$pod = $client->pods()->findByName('my-nginx');

$status = $pod->getStatus();
echo "Pod IP:   " . ($status['podIP'] ?? 'N/A') . "\n";
echo "Host IP:  " . ($status['hostIP'] ?? 'N/A') . "\n";

// All pod IPs (for multi-homed pods)
foreach ($status['podIPs'] ?? [] as $ipEntry) {
    echo "IP: " . $ipEntry['ip'] . "\n";
}
```

Resource Summary

```php
// Print a kubectl get pods-like summary
function printPodTable(Client $client): void
{
    $pods = $client->pods()->find();

    printf("%-45s %-10s %-5s %-12s %-6s\n", 'NAME', 'STATUS', 'READY', 'NODE',
'AGE');
    printf("%s\n", str_repeat('-', 85));

    foreach ($pods as $pod) {
        $name     = $pod->getMetadata('name');
        $phase    = $pod->getStatus()['phase'] ?? 'Unknown';
        $node     = $pod->getSpec()['nodeName'] ?? '-';
        $statuses = $pod->getStatus()['containerStatuses'] ?? [];
        $ready    = count(array_filter($statuses, fn($s) => $s['ready']));
        $total    = count($statuses);

        $start = $pod->getStatus()['startTime'] ?? null;
        $age   = $start ? human_age(new DateTime($start)) : '-';

        printf("%-45s %-10s %-5s %-12s %-6s\n", $name, $phase, "$ready/$total",
$node, $age);
    }
}
```

```
function human_age(DateTime $start): string
{
    $diff = (new DateTime())->diff($start);
    if ($diff->days > 0) return $diff->days . 'd';
    if ($diff->h > 0)    return $diff->h . 'h';
    return $diff->i . 'm';
}
```

Chapter 5: Deployments & ReplicaSets

What Deployments Give You

Feature	How
Desired pod count	`replicas: 3` — always maintained
Rolling updates	New pods before old ones are removed
Rollback	`kubectl rollout undo` — or via API
Pause / resume	Pause rollout mid-way for validation
History	Last N revisions stored in etcd

Listing Deployments

php

```
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$deployments = $client->deployments()->find();

foreach ($deployments as $dep) {
    $name      = $dep->getMetadata('name');
    $desired   = $dep->getSpec()['replicas'] ?? 0;
    $ready     = $dep->getStatus()['readyReplicas'] ?? 0;
    $available = $dep->getStatus()['availableReplicas'] ?? 0;
```

```
    printf("%-40s %d/%d  available=%d\n", $name, $ready, $desired, $available);
}
```

Getting a Single Deployment

```php
$dep = $client->deployments()->findByName('my-app');

echo "Image: " . $dep->getSpec()['template']['spec']['containers'][0]['image']
. "\n";
echo "Replicas desired:   " . $dep->getSpec()['replicas'] . "\n";
echo "Replicas ready:     " . ($dep->getStatus()['readyReplicas'] ?? 0) . "\n";
echo "Replicas available: " . ($dep->getStatus()['availableReplicas'] ?? 0) .
"\n";
```

Creating a Deployment

```php
use Maclof\Kubernetes\Models\Deployment;

$deployment = new Deployment([
    'metadata' => [
        'name'      => 'my-app',
        'namespace' => 'default',
        'labels'    => ['app' => 'my-app'],
    ],
    'spec' => [
        'replicas' => 3,
        'selector' => [
            'matchLabels' => ['app' => 'my-app'],
        ],
        'strategy' => [
            'type' => 'RollingUpdate',
            'rollingUpdate' => [
                'maxUnavailable' => 1,
                'maxSurge'       => 1,
            ],
        ],
        'template' => [
```

```
            'metadata' => [
                'labels' => ['app' => 'my-app'],
            ],
            'spec' => [
                'containers' => [
                    [
                        'name'  => 'my-app',
                        'image' => 'my-registry/my-app:1.0.0',
                        'ports' => [['containerPort' => 8080]],
                        'resources' => [
                            'requests' => ['memory' => '128Mi', 'cpu' =>
'100m'],
                            'limits'   => ['memory' => '256Mi', 'cpu' =>
'500m'],
                        ],
                        'readinessProbe' => [
                            'httpGet' => ['path' => '/health', 'port' => 8080],
                            'initialDelaySeconds' => 5,
                            'periodSeconds'       => 10,
                        ],
                        'livenessProbe' => [
                            'httpGet' => ['path' => '/health', 'port' => 8080],
                            'initialDelaySeconds' => 15,
                            'periodSeconds'       => 20,
                        ],
                    ],
                ],
            ],
        ],
    ],
]);

$client->deployments()->create($deployment);
echo "Deployment created\n";
```

Scaling a Deployment

php

```
// Read current deployment
$dep = $client->deployments()->findByName('my-app');

// Modify replica count
$spec = $dep->getSpec();
```

```php
$spec['replicas'] = 5;

// Patch the deployment
$client->deployments()->patch($dep, [
    'spec' => ['replicas' => 5],
]);

echo "Scaled to 5 replicas\n";
```

Scale via Dedicated Scale Subresource

```php
use Maclof\Kubernetes\Models\Scale;

$scale = new Scale([
    'metadata' => ['name' => 'my-app', 'namespace' => 'default'],
    'spec'     => ['replicas' => 10],
]);

$client->deployments()->scale('my-app', $scale);
```

Updating a Deployment (Rolling Update)

```php
// Update image to trigger rolling update
$dep = $client->deployments()->findByName('my-app');

$dep->setAttribute('spec', array_merge_recursive($dep->getSpec(), [
    'template' => [
        'spec' => [
            'containers' => [
                [
                    'name'  => 'my-app',
                    'image' => 'my-registry/my-app:2.0.0',  // new version
                ],
            ],
        ],
    ],
]));

$client->deployments()->update($dep);
echo "Rolling update triggered to v2.0.0\n";
```

Patch Method (Safer)

```php
$client->deployments()->patch($dep, [
    'spec' => [
        'template' => [
            'spec' => [
                'containers' => [[
                    'name'  => 'my-app',
                    'image' => 'my-registry/my-app:2.0.0',
                ]],
            ],
        ],
    ],
]);
```

Waiting for Rollout Completion

```php
function waitForDeployment(Client $client, string $name, int $timeout = 120):
bool
{
    $start = time();
    while (time() - $start < $timeout) {
        $dep = $client->deployments()->findByName($name);

        $desired   = $dep->getSpec()['replicas'] ?? 0;
        $ready     = $dep->getStatus()['readyReplicas'] ?? 0;
        $updated   = $dep->getStatus()['updatedReplicas'] ?? 0;
        $available = $dep->getStatus()['availableReplicas'] ?? 0;

        echo "Rollout: updated=$updated ready=$ready available=$available
desired=$desired\n";

        if ($updated === $desired && $ready === $desired && $available ===
$desired) {
            return true;
        }

        sleep(3);
    }
    return false;
}
```

```php
// Usage
$client->deployments()->patch($dep, ['spec' => ['template' => ['spec' =>
['containers' => [['name' => 'app', 'image' => 'app:2.0']]]]]]);

if (waitForDeployment($client, 'my-app')) {
    echo "Deployment rolled out successfully\n";
} else {
    echo "Deployment timed out — check pod status\n";
}
```

Deleting a Deployment

```php
// Delete by name (also deletes associated ReplicaSets and Pods)
$client->deployments()->deleteByName('my-app');

// With propagation policy
$dep = $client->deployments()->findByName('my-app');
$client->deployments()->delete($dep, [
    'propagationPolicy' => 'Foreground',  // wait for pods to terminate
]);
```

Working with ReplicaSets

ReplicaSets are usually managed by Deployments, but you can interact with them directly:

```php
// List ReplicaSets owned by a deployment
$rsets = $client->replicaSets()->setLabelSelector(['app' => 'my-app'])->find();

foreach ($rsets as $rs) {
    $name    = $rs->getMetadata('name');
    $desired = $rs->getSpec()['replicas'] ?? 0;
    $ready   = $rs->getStatus()['readyReplicas'] ?? 0;

    // Check if this is the active RS (has ready replicas)
    $active = $ready > 0 ? 'ACTIVE' : 'old';
```

```
    echo "[$active] $name — $ready/$desired ready\n";
}
```

Deployment Status Conditions

```php
$dep = $client->deployments()->findByName('my-app');
$conditions = $dep->getStatus()['conditions'] ?? [];

foreach ($conditions as $cond) {
    printf(
        "%-25s %-10s %s\n",
        $cond['type'],
        $cond['status'],
        $cond['message'] ?? ''
    );
}
```

Condition Type	Meaning
`Available`	Minimum replicas are ready
`Progressing`	Rollout is in progress or complete
`ReplicaFailure`	Replica creation failed

Deployment Strategy Comparison

```php
// RollingUpdate (default) — no downtime
'strategy' => [
    'type' => 'RollingUpdate',
    'rollingUpdate' => [
        'maxUnavailable' => '25%',
        'maxSurge'       => '25%',
    ],
],

// Recreate — downtime between old and new
```

```
    'strategy' => [
        'type' => 'Recreate',
    ],
```

Chapter 6: Services & Networking

Service Types

Type	Access	Use Case
`ClusterIP`	Internal only (within cluster)	Microservice-to-microservice
`NodePort`	Node IP + port	Dev/testing, external access
`LoadBalancer`	Cloud load balancer IP	Production internet-facing
`ExternalName`	DNS alias to external service	Connect to external DB by name

---

Listing Services

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$services = $client->services()->find();

foreach ($services as $svc) {
    $name      = $svc->getMetadata('name');
    $type      = $svc->getSpec()['type'] ?? 'ClusterIP';
    $clusterIP = $svc->getSpec()['clusterIP'] ?? 'None';
    $ports     = $svc->getSpec()['ports'] ?? [];

    $portStr = implode(',', array_map(
        fn($p) => ($p['port'] ?? '') . '/' . ($p['protocol'] ?? 'TCP'),
        $ports
    ));

    printf("%-35s %-15s %-16s %s\n", $name, $type, $clusterIP, $portStr);
}
```

Creating a ClusterIP Service

```php
use Maclof\Kubernetes\Models\Service;

$service = new Service([
    'metadata' => [
        'name'      => 'my-app-svc',
        'namespace' => 'default',
        'labels'    => ['app' => 'my-app'],
    ],
    'spec' => [
        'type'     => 'ClusterIP',
        'selector' => ['app' => 'my-app'],
        'ports'    => [
            [
                'name'       => 'http',
                'port'       => 80,
                'targetPort' => 8080,
                'protocol'   => 'TCP',
            ],
        ],
    ],
]);

$client->services()->create($service);
echo "ClusterIP service created\n";
```

Creating a NodePort Service

```php
$service = new Service([
    'metadata' => ['name' => 'my-app-nodeport'],
    'spec' => [
        'type'     => 'NodePort',
        'selector' => ['app' => 'my-app'],
        'ports'    => [
            [
                'port'       => 80,
                'targetPort' => 8080,
                'nodePort'   => 30080,    // 30000–32767
```

```
            ],
        ],
    ],
]);

$client->services()->create($service);
```

Creating a LoadBalancer Service

```php
$service = new Service([
    'metadata' => [
        'name' => 'my-app-lb',
        'annotations' => [
            // GCP: Specify internal LB
            'cloud.google.com/load-balancer-type' => 'Internal',
            // AWS: Specify NLB type
            'service.beta.kubernetes.io/aws-load-balancer-type' => 'nlb',
        ],
    ],
    'spec' => [
        'type'     => 'LoadBalancer',
        'selector' => ['app' => 'my-app'],
        'ports'    => [
            ['port' => 80, 'targetPort' => 8080],
            ['port' => 443, 'targetPort' => 8443],
        ],
    ],
]);

$created = $client->services()->create($service);

// Wait for LoadBalancer IP to be assigned
$svc = waitForLoadBalancerIP($client, 'my-app-lb');
echo "External IP: " . $svc['status']['loadBalancer']['ingress'][0]['ip'] .
"\n";
```

```php
function waitForLoadBalancerIP(Client $client, string $name, int $timeout =
120): array
{
    $start = time();
    while (time() - $start < $timeout) {
```

```
        $svc = $client->services()->findByName($name);
        $ingress = $svc->getStatus()['loadBalancer']['ingress'] ?? [];
        if (!empty($ingress)) {
            return $svc->toArray();
        }
        echo "Waiting for external IP...\n";
        sleep(5);
    }
    throw new RuntimeException("LoadBalancer IP not assigned within timeout");
}
```

Creating an ExternalName Service

```php
$service = new Service([
    'metadata' => ['name' => 'external-db'],
    'spec' => [
        'type'         => 'ExternalName',
        'externalName' => 'my-database.rds.amazonaws.com',
    ],
]);

$client->services()->create($service);
// Now pods can connect to 'external-db' and it resolves to the RDS host
```

Getting Service Details

```php
$svc = $client->services()->findByName('my-app-svc');

echo "Name:       " . $svc->getMetadata('name') . "\n";
echo "ClusterIP:  " . $svc->getSpec()['clusterIP'] . "\n";
echo "Type:       " . $svc->getSpec()['type'] . "\n";

foreach ($svc->getSpec()['ports'] as $port) {
    echo "Port: " . $port['port'] . " → " . $port['targetPort'] . "\n";
}

// External IPs for LoadBalancer
```

```php
$ingress = $svc->getStatus()['loadBalancer']['ingress'] ?? [];
foreach ($ingress as $ing) {
    echo "External: " . ($ing['ip'] ?? $ing['hostname'] ?? 'pending') . "\n";
}
```

Working with Endpoints

Endpoints show which Pod IPs a Service currently routes to:

```php
$endpoints = $client->endpoints()->findByName('my-app-svc');

$subsets = $endpoints->getAttribute('subsets') ?? [];
foreach ($subsets as $subset) {
    $port = $subset['ports'][0]['port'] ?? 'N/A';
    foreach ($subset['addresses'] ?? [] as $addr) {
        echo "Ready: " . $addr['ip'] . ":" . $port
            . " (Pod: " . ($addr['targetRef']['name'] ?? 'N/A') . ")\n";
    }
    foreach ($subset['notReadyAddresses'] ?? [] as $addr) {
        echo "NotReady: " . $addr['ip'] . "\n";
    }
}
```

Deleting a Service

```php
$client->services()->deleteByName('my-app-svc');
```

Headless Service (StatefulSet DNS)

```php
$service = new Service([
    'metadata' => ['name' => 'my-statefulset-headless'],
    'spec' => [
        'clusterIP' => 'None',   // headless — no VIP assigned
```

```
        'selector'  => ['app' => 'my-statefulset'],
        'ports'     => [['port' => 3306]],
    ],
]);

$client->services()->create($service);
// Each pod now has DNS: pod-0.my-statefulset-
headless.default.svc.cluster.local
```

Checking Service → Pod Connectivity

```php
function checkServiceHealth(Client $client, string $serviceName): array
{
    $svc       = $client->services()->findByName($serviceName);
    $endpoints = $client->endpoints()->findByName($serviceName);
    $selector  = $svc->getSpec()['selector'] ?? [];

    // Count pods matching selector
    $pods       = $client->pods()->setLabelSelector($selector)->find();
    $readyPods  = array_filter(iterator_to_array($pods), function ($pod) {
        $statuses = $pod->getStatus()['containerStatuses'] ?? [];
        return !empty(array_filter($statuses, fn($s) => $s['ready']));
    });

    $subsets       = $endpoints->getAttribute('subsets') ?? [];
    $endpointCount = array_sum(array_map(fn($s) => count($s['addresses'] ??
[]), $subsets));

    return [
        'service'          => $serviceName,
        'type'             => $svc->getSpec()['type'],
        'clusterIP'        => $svc->getSpec()['clusterIP'],
        'total_pods'       => count(iterator_to_array($pods)),
        'ready_pods'       => count($readyPods),
        'endpoint_ips'     => $endpointCount,
    ];
}
```

Network Policy Awareness

While PHP client cannot enforce network policies, you can read them:

```php
$netPolicies = $client->networkPolicies()->find();

foreach ($netPolicies as $np) {
    $name        = $np->getMetadata('name');
    $podSelector = json_encode($np->getSpec()['podSelector'] ?? []);
    echo "Policy: $name applies to: $podSelector\n";
}
```

Chapter 7: ConfigMaps & Secrets

ConfigMaps vs Secrets

Feature	ConfigMap	Secret
Data type	Plain text	Base64-encoded
Stored in etcd	Plaintext	Encrypted (if etcd encryption enabled)
Use case	App config, feature flags	Passwords, tokens, TLS certs
Size limit	1 MiB	1 MiB
Mount as	File, env var	File, env var

Working with ConfigMaps

List ConfigMaps

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$configMaps = $client->configMaps()->find();
```

```
foreach ($configMaps as $cm) {
    $name = $cm->getMetadata('name');
    $keys = array_keys($cm->getAttribute('data') ?? []);
    echo "$name: " . implode(', ', $keys) . "\n";
}
```

Get ConfigMap Data

```php
$cm = $client->configMaps()->findByName('app-config');

$data = $cm->getAttribute('data') ?? [];

echo "DB_HOST: " . ($data['DB_HOST'] ?? 'not set') . "\n";
echo "APP_ENV: " . ($data['APP_ENV'] ?? 'not set') . "\n";

// Read as file content (multi-line config)
$nginxConf = $data['nginx.conf'] ?? '';
file_put_contents('/tmp/nginx.conf', $nginxConf);
```

Create a ConfigMap

```php
use Maclof\Kubernetes\Models\ConfigMap;

$configMap = new ConfigMap([
    'metadata' => [
        'name'      => 'app-config',
        'namespace' => 'default',
        'labels'    => ['app' => 'my-app'],
    ],
    'data' => [
        'APP_ENV'       => 'production',
        'DB_HOST'       => 'postgres.default.svc.cluster.local',
        'DB_PORT'       => '5432',
        'CACHE_TTL'     => '3600',
        'nginx.conf'    => "server {\n    listen 80;\n    server_name _;\n
location / { proxy_pass http://app:8080; }\n}\n",
    ],
]);

$client->configMaps()->create($configMap);
echo "ConfigMap created\n";
```

Update a ConfigMap

```php
$cm = $client->configMaps()->findByName('app-config');

$data = $cm->getAttribute('data') ?? [];
$data['CACHE_TTL'] = '7200';   // update a value
$data['NEW_FLAG']  = 'true';   // add a new key

$cm->setAttribute('data', $data);
$client->configMaps()->update($cm);
echo "ConfigMap updated\n";
```

Delete a ConfigMap

```php
$client->configMaps()->deleteByName('app-config');
```

Working with Secrets

> **Security Rule:** Never log, echo, or expose Secret values. Always handle them with care.

List Secrets (Names Only — Never Print Values)

```php
$secrets = $client->secrets()->find();

foreach ($secrets as $secret) {
    $name = $secret->getMetadata('name');
    $type = $secret->getAttribute('type') ?? 'Opaque';
    $keys = array_keys($secret->getAttribute('data') ?? []);
    echo "$name ($type): " . implode(', ', $keys) . "\n";
}
```

Create an Opaque Secret

Kubernetes expects Secret data to be **base64-encoded**:

```php
use Maclof\Kubernetes\Models\Secret;

$secret = new Secret([
```

```
        'metadata' => [
            'name'      => 'db-credentials',
            'namespace' => 'default',
            'labels'    => ['app' => 'my-app'],
        ],
        'type' => 'Opaque',
        'data' => [
            'username' => base64_encode('admin'),
            'password' => base64_encode('super-secret-password'),
            'db_url'   =>
    base64_encode('postgres://admin:pass@postgres:5432/mydb'),
        ],
    ]);

    $client->secrets()->create($secret);
    echo "Secret created\n";
```

Create a Docker Registry Secret

php

```
    $dockerConfig = json_encode([
        'auths' => [
            'registry.example.com' => [
                'username' => 'myuser',
                'password' => 'mypass',
                'email'    => 'ci@example.com',
                'auth'     => base64_encode('myuser:mypass'),
            ],
        ],
    ]);

    $secret = new Secret([
        'metadata' => ['name' => 'registry-credentials'],
        'type'     => 'kubernetes.io/dockerconfigjson',
        'data'     => [
            '.dockerconfigjson' => base64_encode($dockerConfig),
        ],
    ]);

    $client->secrets()->create($secret);
```

Create a TLS Secret

```php
$tlsSecret = new Secret([
    'metadata' => ['name' => 'my-tls-secret'],
    'type'     => 'kubernetes.io/tls',
    'data'     => [
        'tls.crt' => base64_encode(file_get_contents('/path/to/tls.crt')),
        'tls.key' => base64_encode(file_get_contents('/path/to/tls.key')),
    ],
]);

$client->secrets()->create($tlsSecret);
```

Read a Secret Value

```php
$secret = $client->secrets()->findByName('db-credentials');

$data = $secret->getAttribute('data') ?? [];

// Values are base64-encoded in the API response — decode them
$username = base64_decode($data['username'] ?? '');
$password = base64_decode($data['password'] ?? '');

// Use, but don't log the password
$dsn = "pgsql:host=postgres;dbname=mydb;user=$username;password=$password";
$pdo = new PDO($dsn);
```

Update a Secret

```php
$secret = $client->secrets()->findByName('db-credentials');

$data = $secret->getAttribute('data') ?? [];
$data['password'] = base64_encode('new-rotated-password');

$secret->setAttribute('data', $data);
$client->secrets()->update($secret);
echo "Secret rotated\n";
```

Delete a Secret

```php
$client->secrets()->deleteByName('db-credentials');
```

Secret Rotation Pattern

```php
function rotateSecret(Client $client, string $secretName, string $key, string $newValue): void
{
    $secret = $client->secrets()->findByName($secretName);
    $data   = $secret->getAttribute('data') ?? [];

    // Store previous value for rollback
    if (isset($data[$key])) {
        $data[$key . '_previous'] = $data[$key];
    }

    $data[$key] = base64_encode($newValue);
    $secret->setAttribute('data', $data);
    $client->secrets()->update($secret);

    echo "Secret '$secretName' key '$key' rotated successfully\n";
}

rotateSecret($client, 'db-credentials', 'password', 'new-secure-password-v2');
```

Bulk ConfigMap from PHP Array

```php
function syncConfigMap(Client $client, string $name, array $config): void
{
    $data = array_map('strval', $config);  // ensure all values are strings

    try {
        $cm = $client->configMaps()->findByName($name);
        $cm->setAttribute('data', $data);
        $client->configMaps()->update($cm);
        echo "ConfigMap '$name' updated\n";
    } catch (\Exception $e) {
        $cm = new \Maclof\Kubernetes\Models\ConfigMap([
            'metadata' => ['name' => $name],
            'data'     => $data,
        ]);
        $client->configMaps()->create($cm);
        echo "ConfigMap '$name' created\n";
```

```
    }
}

syncConfigMap($client, 'feature-flags', [
    'FEATURE_NEW_UI'    => 'true',
    'FEATURE_DARK_MODE' => 'false',
    'MAX_RETRY'         => '3',
]);
```

ConfigMap from .env File

```php
function configMapFromEnvFile(Client $client, string $name, string $envFilePath): void
{
    $lines = file($envFilePath, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    $data  = [];

    foreach ($lines as $line) {
        if (str_starts_with(trim($line), '#')) continue;
        [$key, $value] = explode('=', $line, 2);
        $data[trim($key)] = trim($value, '"\'');
    }

    syncConfigMap($client, $name, $data);
}

configMapFromEnvFile($client, 'app-env-config', '/path/to/.env.production');
```

Chapter 8: Persistent Volumes & Storage

Storage Architecture

```
Developer defines:    PersistentVolumeClaim (PVC)
Cluster provides:     PersistentVolume (PV)
Cloud provisions:     Disk (EBS, GCE PD, Azure Disk) via StorageClass
```

Object	Who Creates It	Purpose
`StorageClass`	Admin	Defines provisioner and disk type
`PersistentVolume`	Admin or Dynamically	The actual storage resource
`PersistentVolumeClaim`	Developer	Request for storage

Working with StorageClasses

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

// List available StorageClasses
$storageClasses = $client->storageClasses()->find();

foreach ($storageClasses as $sc) {
    $name        = $sc->getMetadata('name');
    $provisioner = $sc->getAttribute('provisioner');
    $isDefault   = ($sc->getMetadata('annotations')
['storageclass.kubernetes.io/is-default-class'] ?? 'false') === 'true';
    $reclaimPolicy = $sc->getAttribute('reclaimPolicy') ?? 'Delete';

    printf("%-30s %-40s %-10s %s\n",
        $name,
        $provisioner,
        $reclaimPolicy,
        $isDefault ? '(default)' : ''
    );
}
```

Create a StorageClass

```php
use Maclof\Kubernetes\Models\StorageClass;

$sc = new StorageClass([
    'metadata' => [
        'name'        => 'fast-ssd',
        'annotations' => [
```

```
            'storageclass.kubernetes.io/is-default-class' => 'false',
        ],
    ],
    'provisioner'       => 'kubernetes.io/gce-pd',
    'reclaimPolicy'     => 'Retain',
    'volumeBindingMode' => 'WaitForFirstConsumer',
    'parameters'        => [
        'type'             => 'pd-ssd',
        'replication-type'  => 'regional-pd',
        'zones'            => 'us-central1-a,us-central1-b',
    ],
]);

$client->storageClasses()->create($sc);
```

Working with PersistentVolumeClaims (PVC)

List PVCs

```php
$pvcs = $client->persistentVolumeClaims()->find();

foreach ($pvcs as $pvc) {
    $name         = $pvc->getMetadata('name');
    $status       = $pvc->getStatus()['phase'] ?? 'Unknown';
    $capacity     = $pvc->getStatus()['capacity']['storage'] ?? 'N/A';
    $storageClass = $pvc->getSpec()['storageClassName'] ?? '-';
    $accessModes  = implode(',', $pvc->getSpec()['accessModes'] ?? []);

    printf("%-30s %-10s %-10s %-20s %s\n",
        $name, $status, $capacity, $storageClass, $accessModes);
}
```

Create a PVC

```php
use Maclof\Kubernetes\Models\PersistentVolumeClaim;

$pvc = new PersistentVolumeClaim([
    'metadata' => [
        'name'      => 'my-data-pvc',
        'namespace' => 'default',
```

```
            'labels'    => ['app' => 'my-app'],
        ],
        'spec' => [
            'accessModes'      => ['ReadWriteOnce'],
            'storageClassName' => 'standard',
            'resources'        => [
                'requests' => ['storage' => '10Gi'],
            ],
        ],
    ]);

    $created = $client->persistentVolumeClaims()->create($pvc);
    echo "PVC created: " . $created->getMetadata('name') . "\n";

    // Wait for PVC to be Bound
    $pvc = waitForPVCBound($client, 'my-data-pvc');
    echo "PVC bound to PV: " . $pvc->getSpec()['volumeName'] . "\n";
```

```php
function waitForPVCBound(Client $client, string $pvcName, int $timeout = 60):
mixed
{
    $start = time();
    while (time() - $start < $timeout) {
        $pvc    = $client->persistentVolumeClaims()->findByName($pvcName);
        $phase  = $pvc->getStatus()['phase'] ?? 'Pending';

        if ($phase === 'Bound') return $pvc;

        echo "PVC phase: $phase — waiting...\n";
        sleep(3);
    }
    throw new RuntimeException("PVC not bound within $timeout seconds");
}
```

Create a PVC with Specific Access Mode

```php
// ReadWriteOnce — single node read/write (most disks)
// ReadOnlyMany  — multiple nodes read-only (NFS, etc.)
// ReadWriteMany — multiple nodes read/write (NFS, CephFS)

$sharedPvc = new PersistentVolumeClaim([
    'metadata' => ['name' => 'shared-data'],
    'spec' => [
        'accessModes'      => ['ReadWriteMany'],
```

```
        'storageClassName' => 'nfs-client',
        'resources'        => ['requests' => ['storage' => '50Gi']],
    ],
]);

$client->persistentVolumeClaims()->create($sharedPvc);
```

Working with PersistentVolumes (PV)

List PVs

```php
$pvs = $client->persistentVolumes()->find();

foreach ($pvs as $pv) {
    $name         = $pv->getMetadata('name');
    $capacity     = $pv->getSpec()['capacity']['storage'] ?? 'N/A';
    $accessModes  = implode(',', $pv->getSpec()['accessModes'] ?? []);
    $phase        = $pv->getStatus()['phase'] ?? 'Unknown';
    $claim        = $pv->getSpec()['claimRef']['name'] ?? '-';

    printf("%-30s %-8s %-20s %-10s %s\n",
        $name, $capacity, $accessModes, $phase, $claim);
}
```

Create a PV (Static Provisioning)

```php
use Maclof\Kubernetes\Models\PersistentVolume;

$pv = new PersistentVolume([
    'metadata' => [
        'name'   => 'my-nfs-pv',
        'labels' => ['type' => 'nfs'],
    ],
    'spec' => [
        'capacity'        => ['storage' => '100Gi'],
        'accessModes'     => ['ReadWriteMany'],
        'reclaimPolicy'   => 'Retain',
        'storageClassName' => 'nfs',
        'nfs' => [
            'server' => '10.0.0.5',
```

```
            'path'   => '/exported/data',
        ],
    ],
]);

$client->persistentVolumes()->create($pv);
```

Mounting a PVC in a Deployment

```php
use Maclof\Kubernetes\Models\Deployment;

$deployment = new Deployment([
    'metadata' => ['name' => 'app-with-storage'],
    'spec' => [
        'replicas' => 1,
        'selector' => ['matchLabels' => ['app' => 'app-with-storage']],
        'template' => [
            'metadata' => ['labels' => ['app' => 'app-with-storage']],
            'spec' => [
                'containers' => [[
                    'name'  => 'app',
                    'image' => 'myapp:1.0',
                    'volumeMounts' => [
                        [
                            'name'      => 'data-volume',
                            'mountPath' => '/var/data',
                        ],
                    ],
                ]],
                'volumes' => [
                    [
                        'name' => 'data-volume',
                        'persistentVolumeClaim' => [
                            'claimName' => 'my-data-pvc',
                        ],
                    ],
                ],
            ],
        ],
    ],
]);
```

```php
$client->deployments()->create($deployment);
```

Storage Capacity Check

```php
function listStorageUsage(Client $client): void
{
    $pvcs = $client->persistentVolumeClaims()->find();

    $total = 0;
    foreach ($pvcs as $pvc) {
        $name     = $pvc->getMetadata('name');
        $ns       = $pvc->getMetadata('namespace');
        $phase    = $pvc->getStatus()['phase'] ?? '-';
        $size     = $pvc->getSpec()['resources']['requests']['storage'] ?? '0';

        printf("%-30s %-15s %-10s %s\n", $name, $ns, $phase, $size);

        // Parse sizes (basic)
        if (str_ends_with($size, 'Gi')) {
            $total += (int) $size;
        }
    }

    echo "\nTotal storage requested: {$total}Gi\n";
}
```

Deleting a PVC

```php
// PVC must not be in use by any pod
$client->persistentVolumeClaims()->deleteByName('my-data-pvc');

// The PV behavior depends on reclaimPolicy:
// - Delete: PV and underlying disk are deleted
// - Retain: PV remains, must be manually reclaimed
// - Recycle: PV is scrubbed (deprecated)
```

Chapter 9: Namespaces & RBAC

Why Namespaces & RBAC?

Problem	Solution
Isolate teams (dev/staging/prod)	Separate Namespaces
Prevent team A from deleting team B's resources	RBAC Roles
Give CI pipeline only create/read pod access	ServiceAccount + RoleBinding
Allow dashboard to read cluster-wide resources	ClusterRole + ClusterRoleBinding

---

Working with Namespaces

List Namespaces

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();
$client->setNamespace('');  // target all namespaces for cluster-scoped resources

$namespaces = $client->namespaces()->find();

foreach ($namespaces as $ns) {
    $name   = $ns->getMetadata('name');
    $status = $ns->getStatus()['phase'] ?? 'Unknown';
    echo "[$status] $name\n";
}
```

Create a Namespace

```php
use Maclof\Kubernetes\Models\NamespaceModel;

$namespace = new NamespaceModel([
    'metadata' => [
```

```
            'name' => 'team-alpha',
            'labels' => [
                'team'        => 'alpha',
                'environment' => 'development',
            ],
            'annotations' => [
                'owner' => 'alice@example.com',
            ],
        ],
    ]);

    $client->namespaces()->create($namespace);
    echo "Namespace 'team-alpha' created\n";
```

Delete a Namespace

```php
    // Deleting a namespace deletes ALL resources inside it
    $client->namespaces()->deleteByName('team-alpha');
    // Termination takes time — pods are cleaned up first
```

Watch Namespace Status (Terminating)

```php
    function waitForNamespaceGone(Client $client, string $name, int $timeout =
    120): void
    {
        $start = time();
        while (time() - $start < $timeout) {
            $ns = $client->namespaces()->findByName($name);
            if ($ns === null) {
                echo "Namespace '$name' fully deleted\n";
                return;
            }
            $phase = $ns->getStatus()['phase'] ?? 'Active';
            echo "Namespace phase: $phase — waiting...\n";
            sleep(3);
        }
        throw new RuntimeException("Namespace '$name' still terminating after
    $timeout seconds");
    }
```

Working with ServiceAccounts

```php
use Maclof\Kubernetes\Models\ServiceAccount;

// Create a ServiceAccount for a CI pipeline
$sa = new ServiceAccount([
    'metadata' => [
        'name'      => 'ci-pipeline',
        'namespace' => 'default',
        'labels'    => ['purpose' => 'ci'],
        'annotations' => [
            'description' => 'ServiceAccount for GitHub Actions CI pipeline',
        ],
    ],
]);

$client->serviceAccounts()->create($sa);

// Generate a token for it (K8s 1.24+)
// Use kubectl: kubectl create token ci-pipeline
```

List ServiceAccounts

```php
$serviceAccounts = $client->serviceAccounts()->find();

foreach ($serviceAccounts as $sa) {
    echo $sa->getMetadata('name') . "\n";
}
```

Working with Roles (Namespaced)

A **Role** grants permissions within a single namespace.

```php
use Maclof\Kubernetes\Models\Role;

$role = new Role([
    'metadata' => [
        'name'      => 'pod-reader',
        'namespace' => 'default',
    ],
```

```
    'rules' => [
        [
            'apiGroups' => [''],           // '' = core API group
            'resources' => ['pods', 'pods/log'],
            'verbs'     => ['get', 'list', 'watch'],
        ],
    ],
]);

$client->roles()->create($role);
```

Deployment Manager Role

```php
$role = new Role([
    'metadata' => ['name' => 'deployment-manager', 'namespace' =>
'production'],
    'rules' => [
        [
            'apiGroups' => ['apps'],
            'resources' => ['deployments', 'replicasets'],
            'verbs'     => ['get', 'list', 'watch', 'create', 'update',
'patch'],
        ],
        [
            'apiGroups' => [''],
            'resources' => ['pods', 'services', 'configmaps'],
            'verbs'     => ['get', 'list', 'watch'],
        ],
    ],
]);

$client->setNamespace('production');
$client->roles()->create($role);
```

Working with ClusterRoles (Cluster-Wide)

```php
use Maclof\Kubernetes\Models\ClusterRole;

$clusterRole = new ClusterRole([
    'metadata' => ['name' => 'cluster-reader'],
```

```
    'rules' => [
        [
            'apiGroups' => ['', 'apps', 'batch'],
            'resources' => ['*'],
            'verbs'     => ['get', 'list', 'watch'],
        ],
        [
            'apiGroups' => [''],
            'resources' => ['nodes', 'namespaces', 'persistentvolumes'],
            'verbs'     => ['get', 'list'],
        ],
    ],
]);

$client->clusterRoles()->create($clusterRole);
```

Working with RoleBindings

Bind a Role to a user or ServiceAccount within a namespace:

php

```
use Maclof\Kubernetes\Models\RoleBinding;

$roleBinding = new RoleBinding([
    'metadata' => [
        'name'      => 'ci-pipeline-binding',
        'namespace' => 'default',
    ],
    'subjects' => [
        [
            'kind'      => 'ServiceAccount',
            'name'      => 'ci-pipeline',
            'namespace' => 'default',
        ],
    ],
    'roleRef' => [
        'apiGroup' => 'rbac.authorization.k8s.io',
        'kind'     => 'Role',
        'name'     => 'deployment-manager',
    ],
]);
```

```php
$client->roleBindings()->create($roleBinding);
echo "RoleBinding created: ci-pipeline → deployment-manager\n";
```

Bind to a User (not ServiceAccount)

```php
$roleBinding = new RoleBinding([
    'metadata' => ['name' => 'alice-pod-reader', 'namespace' => 'team-alpha'],
    'subjects' => [
        [
            'kind'     => 'User',
            'name'     => 'alice@example.com',
            'apiGroup' => 'rbac.authorization.k8s.io',
        ],
    ],
    'roleRef' => [
        'apiGroup' => 'rbac.authorization.k8s.io',
        'kind'     => 'Role',
        'name'     => 'pod-reader',
    ],
]);

$client->roleBindings()->create($roleBinding);
```

Working with ClusterRoleBindings

```php
use Maclof\Kubernetes\Models\ClusterRoleBinding;

$crb = new ClusterRoleBinding([
    'metadata' => ['name' => 'dashboard-cluster-reader'],
    'subjects' => [
        [
            'kind'      => 'ServiceAccount',
            'name'      => 'dashboard',
            'namespace' => 'kube-system',
        ],
    ],
    'roleRef' => [
        'apiGroup' => 'rbac.authorization.k8s.io',
        'kind'     => 'ClusterRole',
        'name'     => 'cluster-reader',
```

```
        ],
    ]);

    $client->clusterRoleBindings()->create($crb);
```

Multi-Tenant Namespace Provisioning

Automate creating everything needed for a new team:

```php
function provisionTenantNamespace(Client $client, string $teamName, string $ownerEmail): void
{
    $ns = $teamName . '-workspace';

    // 1. Create namespace
    $client->namespaces()->create(new NamespaceModel([
        'metadata' => [
            'name'        => $ns,
            'labels'      => ['team' => $teamName],
            'annotations' => ['owner' => $ownerEmail],
        ],
    ]));

    $client->setNamespace($ns);

    // 2. Create ServiceAccount
    $client->serviceAccounts()->create(new ServiceAccount([
        'metadata' => ['name' => "$teamName-sa", 'namespace' => $ns],
    ]));

    // 3. Create Role
    $client->roles()->create(new Role([
        'metadata' => ['name' => "$teamName-role", 'namespace' => $ns],
        'rules' => [
            ['apiGroups' => ['', 'apps', 'batch'], 'resources' => ['*'], 'verbs' => ['*']],
        ],
    ]));

    // 4. Bind SA to Role
    $client->roleBindings()->create(new RoleBinding([
        'metadata' => ['name' => "$teamName-binding", 'namespace' => $ns],
```

```
        'subjects' => [['kind' => 'ServiceAccount', 'name' => "$teamName-sa",
'namespace' => $ns]],
        'roleRef'  => ['apiGroup' => 'rbac.authorization.k8s.io', 'kind' =>
'Role', 'name' => "$teamName-role"],
    ]));

    echo "Namespace '$ns' provisioned for team '$teamName'\n";
}
```

Listing Roles and Bindings

php

```php
// List all roles in current namespace
$roles = $client->roles()->find();
foreach ($roles as $role) {
    echo "Role: " . $role->getMetadata('name') . "\n";
    foreach ($role->getAttribute('rules') as $rule) {
        echo "  Resources: " . implode(', ', $rule['resources'])
            . " | Verbs: " . implode(', ', $rule['verbs']) . "\n";
    }
}

// List all role bindings
$bindings = $client->roleBindings()->find();
foreach ($bindings as $binding) {
    $subjects = array_map(fn($s) => $s['name'], $binding-
>getAttribute('subjects'));
    echo $binding->getMetadata('name') . " → " . implode(', ', $subjects) .
"\n";
}
```

Chapter 10: Ingress

What is Ingress?

Ingress is the HTTP/HTTPS layer router for Kubernetes. Instead of creating a LoadBalancer per service, one Ingress controller handles all external traffic and routes by hostname or path.

```
Internet
   |
   ▼
```

```
Ingress Controller (nginx, traefik, etc.)
    ├── /api/*       → api-service:80
    ├── /app/*       → frontend-service:80
    └── admin.host   → admin-service:8080
```

Prerequisites

```
# Install nginx ingress controller
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/controller-v1.9.4/deploy/static/provider/cloud/deploy.yaml

# For minikube
minikube addons enable ingress
```

Listing Ingresses

php

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$ingresses = $client->ingresses()->find();

foreach ($ingresses as $ing) {
    $name  = $ing->getMetadata('name');
    $rules = $ing->getSpec()['rules'] ?? [];

    foreach ($rules as $rule) {
        $host  = $rule['host'] ?? '*';
        $paths = $rule['http']['paths'] ?? [];
        foreach ($paths as $path) {
            $backend = $path['backend']['service']['name'] ?? 'N/A';
            $port    = $path['backend']['service']['port']['number'] ?? 'N/A';
            $p       = $path['path'] ?? '/';
            printf("%-20s %-25s %-10s → %s:%s\n", $name, $host, $p, $backend,
$port);
        }
```

```
    }
}
```

Creating a Basic Ingress (Path-based Routing)

```php
use Maclof\Kubernetes\Models\Ingress;

$ingress = new Ingress([
    'metadata' => [
        'name'      => 'my-app-ingress',
        'namespace' => 'default',
        'annotations' => [
            'nginx.ingress.kubernetes.io/rewrite-target' => '/',
        ],
    ],
    'spec' => [
        'ingressClassName' => 'nginx',
        'rules' => [
            [
                'host' => 'myapp.example.com',
                'http' => [
                    'paths' => [
                        [
                            'path'     => '/api',
                            'pathType' => 'Prefix',
                            'backend'  => [
                                'service' => [
                                    'name' => 'api-service',
                                    'port' => ['number' => 80],
                                ],
                            ],
                        ],
                        [
                            'path'     => '/',
                            'pathType' => 'Prefix',
                            'backend'  => [
                                'service' => [
                                    'name' => 'frontend-service',
                                    'port' => ['number' => 80],
                                ],
                            ],
                        ],
```

```
                    ],
                ],
            ],
        ],
    ],
]);

$client->ingresses()->create($ingress);
echo "Ingress created\n";
```

Creating an Ingress with TLS

```php
$ingress = new Ingress([
    'metadata' => [
        'name'      => 'my-app-tls-ingress',
        'namespace' => 'default',
        'annotations' => [
            'cert-manager.io/cluster-issuer'            => 'letsencrypt-prod',
            'nginx.ingress.kubernetes.io/ssl-redirect'  => 'true',
        ],
    ],
    'spec' => [
        'ingressClassName' => 'nginx',
        'tls' => [
            [
                'hosts'      => ['myapp.example.com', 'api.example.com'],
                'secretName' => 'myapp-tls-secret',
            ],
        ],
        'rules' => [
            [
                'host' => 'myapp.example.com',
                'http' => [
                    'paths' => [[
                        'path'     => '/',
                        'pathType' => 'Prefix',
                        'backend'  => ['service' => ['name' => 'frontend',
'port' => ['number' => 80]]],
                    ]],
                ],
            ],
            [
```

```
                'host' => 'api.example.com',
                'http' => [
                    'paths' => [[
                        'path'     => '/',
                        'pathType' => 'Prefix',
                        'backend'  => ['service' => ['name' => 'api', 'port' =>
['number' => 80]]],
                    ]],
                ],
            ],
        ],
    ],
]);

$client->ingresses()->create($ingress);
```

Multi-Host Ingress

```php
function createMultiHostIngress(Client $client, array $routes): void
{
    // $routes format: ['host.example.com' => ['path' => 'service-name:port']]
    $rules = [];

    foreach ($routes as $host => $paths) {
        $httpPaths = [];
        foreach ($paths as $path => $backend) {
            [$svcName, $port] = explode(':', $backend);
            $httpPaths[] = [
                'path'     => $path,
                'pathType' => 'Prefix',
                'backend'  => ['service' => ['name' => $svcName, 'port' =>
['number' => (int)$port]]],
            ];
        }
        $rules[] = ['host' => $host, 'http' => ['paths' => $httpPaths]];
    }

    $ingress = new Ingress([
        'metadata' => ['name' => 'multi-host-ingress'],
        'spec'     => ['ingressClassName' => 'nginx', 'rules' => $rules],
    ]);
```

```
    $client->ingresses()->create($ingress);
}

createMultiHostIngress($client, [
    'app.example.com'    => ['/' => 'frontend:80', '/api' => 'api:8080'],
    'admin.example.com'  => ['/' => 'admin-panel:3000'],
    'metrics.example.com' => ['/' => 'grafana:3000'],
]);
```

Adding Annotations (nginx-specific)

```php
$annotations = [
    // Rate limiting
    'nginx.ingress.kubernetes.io/limit-rps'       => '10',
    'nginx.ingress.kubernetes.io/limit-connections' => '20',

    // Auth
    'nginx.ingress.kubernetes.io/auth-type'       => 'basic',
    'nginx.ingress.kubernetes.io/auth-secret'     => 'basic-auth',

    // Proxy settings
    'nginx.ingress.kubernetes.io/proxy-body-size'    => '50m',
    'nginx.ingress.kubernetes.io/proxy-read-timeout' => '300',

    // CORS
    'nginx.ingress.kubernetes.io/enable-cors'     => 'true',
    'nginx.ingress.kubernetes.io/cors-allow-origin' => 'https://myapp.com',

    // Redirect HTTP to HTTPS
    'nginx.ingress.kubernetes.io/ssl-redirect'    => 'true',
    'nginx.ingress.kubernetes.io/force-ssl-redirect' => 'true',
];

$ingress->setAttribute('metadata', array_merge(
    $ingress->getMetadata(),
    ['annotations' => $annotations]
));
```

Getting Ingress Address

```php
$ing = $client->ingresses()->findByName('my-app-ingress');

$lbStatus = $ing->getStatus()['loadBalancer']['ingress'] ?? [];
foreach ($lbStatus as $entry) {
    echo "Address: " . ($entry['ip'] ?? $entry['hostname'] ?? 'pending') .
"\n";
}

// Rules summary
foreach ($ing->getSpec()['rules'] ?? [] as $rule) {
    echo "Host: " . ($rule['host'] ?? '*') . "\n";
}
```

Updating an Ingress

```php
$ing = $client->ingresses()->findByName('my-app-ingress');

$spec = $ing->getSpec();
$spec['rules'][] = [
    'host' => 'new-service.example.com',
    'http' => ['paths' => [[
        'path'     => '/',
        'pathType' => 'Prefix',
        'backend'  => ['service' => ['name' => 'new-service', 'port' =>
['number' => 80]]],
    ]]],
];

$ing->setAttribute('spec', $spec);
$client->ingresses()->update($ing);
echo "Ingress updated with new host\n";
```

Deleting an Ingress

```php
$client->ingresses()->deleteByName('my-app-ingress');
```

Path Type Reference

PathType	Behavior
`Exact`	Matches the exact path only
`Prefix`	Matches path prefix
`ImplementationSpecific`	Controller-specific behavior

Chapter 11: StatefulSets & DaemonSets

StatefulSets vs Deployments

Feature	Deployment	StatefulSet
Pod names	`app-7d6f8-xyz` (random)	`app-0`, `app-1`, `app-2` (stable)
Pod DNS	Random	`pod-0.headless-svc.ns.svc.cluster.local`
Scaling order	Parallel	Ordered (0 → 1 → 2)
Storage	Shared or ephemeral	Each pod gets its own PVC (volumeClaimTemplates)
Use case	Stateless apps	Databases, queues, distributed systems

Working with StatefulSets

List StatefulSets

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$statefulSets = $client->statefulSets()->find();

foreach ($statefulSets as $sts) {
    $name     = $sts->getMetadata('name');
    $desired  = $sts->getSpec()['replicas'] ?? 0;
    $ready    = $sts->getStatus()['readyReplicas'] ?? 0;
    $current  = $sts->getStatus()['currentReplicas'] ?? 0;

    printf("%-30s %d/%d  current=%d\n", $name, $ready, $desired, $current);
}
```

Create a StatefulSet (MySQL example)

```php
use Maclof\Kubernetes\Models\StatefulSet;
use Maclof\Kubernetes\Models\Service;

// First, create the headless service for stable DNS
$headlessService = new Service([
    'metadata' => ['name' => 'mysql-headless', 'namespace' => 'default'],
    'spec' => [
        'clusterIP' => 'None',
        'selector'  => ['app' => 'mysql'],
        'ports'     => [['port' => 3306, 'name' => 'mysql']],
    ],
]);
$client->services()->create($headlessService);

// Then create the StatefulSet
$statefulSet = new StatefulSet([
    'metadata' => [
        'name'      => 'mysql',
        'namespace' => 'default',
    ],
    'spec' => [
        'serviceName' => 'mysql-headless',
        'replicas'    => 3,
        'selector'    => ['matchLabels' => ['app' => 'mysql']],
        'template'    => [
            'metadata' => ['labels' => ['app' => 'mysql']],
            'spec'     => [
```

```
                'containers' => [[
                    'name'  => 'mysql',
                    'image' => 'mysql:8.0',
                    'env'   => [
                        ['name' => 'MYSQL_ROOT_PASSWORD', 'valueFrom' => [
                            'secretKeyRef' => ['name' => 'mysql-secret', 'key'
=> 'root-password'],
                        ]],
                    ],
                    'ports' => [['containerPort' => 3306, 'name' => 'mysql']],
                    'volumeMounts' => [
                        ['name' => 'data', 'mountPath' => '/var/lib/mysql'],
                    ],
                    'resources' => [
                        'requests' => ['memory' => '512Mi', 'cpu' => '250m'],
                        'limits'   => ['memory' => '1Gi', 'cpu' => '500m'],
                    ],
                    'readinessProbe' => [
                        'exec' => ['command' => ['bash', '-c', 'mysqladmin ping
-h 127.0.0.1']],
                        'initialDelaySeconds' => 20,
                        'periodSeconds'       => 10,
                    ],
                ]],
            ],
        ],
        'volumeClaimTemplates' => [
            [
                'metadata' => ['name' => 'data'],
                'spec'     => [
                    'accessModes'      => ['ReadWriteOnce'],
                    'storageClassName' => 'standard',
                    'resources'        => ['requests' => ['storage' =>
'20Gi']],
                ],
            ],
        ],
    ],
]);

$client->statefulSets()->create($statefulSet);
echo "MySQL StatefulSet created with 3 replicas\n";
// Pods: mysql-0, mysql-1, mysql-2
// DNS:  mysql-0.mysql-headless.default.svc.cluster.local
```

Scaling a StatefulSet

```php
$client->statefulSets()->patch(
    $client->statefulSets()->findByName('mysql'),
    ['spec' => ['replicas' => 5]]
);
// Scales up ordered: mysql-3, then mysql-4
```

StatefulSet Pod DNS Names

```php
function getStatefulSetPodDNS(string $podName, string $serviceName, string $namespace = 'default'): string
{
    return "$podName.$serviceName.$namespace.svc.cluster.local";
}

// MySQL cluster primary/replica discovery
$sts    = $client->statefulSets()->findByName('mysql');
$count  = $sts->getSpec()['replicas'] ?? 0;
$svcName = $sts->getSpec()['serviceName'];

for ($i = 0; $i < $count; $i++) {
    $dns = getStatefulSetPodDNS("mysql-$i", $svcName);
    echo "Replica $i: $dns\n";
}
```

Update Strategy for StatefulSets

```php
// RollingUpdate (default): pods updated one at a time, from highest to lowest index
// OnDelete: pods only updated when manually deleted

$sts = $client->statefulSets()->findByName('mysql');
$client->statefulSets()->patch($sts, [
    'spec' => [
        'updateStrategy' => [
            'type' => 'RollingUpdate',
            'rollingUpdate' => [
                'partition' => 2,  // only update pods with index >= 2 (canary)
            ],
```

```
            ],
            'template' => [
                'spec' => [
                    'containers' => [['name' => 'mysql', 'image' => 'mysql:8.1']],
                ],
            ],
        ],
    ]);
```

Working with DaemonSets

A **DaemonSet** runs exactly one pod per node. Use for:

- Log collectors (Fluentd, Filebeat)
- Monitoring agents (Prometheus node-exporter, Datadog)
- Network plugins (CNI agents)
- Security scanners

List DaemonSets

```php
$daemonSets = $client->daemonSets()->find();

foreach ($daemonSets as $ds) {
    $name      = $ds->getMetadata('name');
    $desired   = $ds->getStatus()['desiredNumberScheduled'] ?? 0;
    $ready     = $ds->getStatus()['numberReady'] ?? 0;
    $available = $ds->getStatus()['numberAvailable'] ?? 0;

    printf("%-35s desired=%d  ready=%d  available=%d\n",
        $name, $desired, $ready, $available);
}
```

Create a DaemonSet (log collector)

```php
use Maclof\Kubernetes\Models\DaemonSet;

$daemonSet = new DaemonSet([
    'metadata' => [
        'name'      => 'fluentd',
```

```
            'namespace' => 'kube-system',
            'labels'    => ['app' => 'fluentd'],
        ],
        'spec' => [
            'selector' => ['matchLabels' => ['app' => 'fluentd']],
            'updateStrategy' => [
                'type' => 'RollingUpdate',
                'rollingUpdate' => ['maxUnavailable' => 1],
            ],
            'template' => [
                'metadata' => ['labels' => ['app' => 'fluentd']],
                'spec' => [
                    'tolerations' => [
                        ['key' => 'node-role.kubernetes.io/control-plane', 'effect'
    => 'NoSchedule'],
                    ],
                    'containers' => [[
                        'name'  => 'fluentd',
                        'image' => 'fluent/fluentd-kubernetes-daemonset:v1.16',
                        'resources' => [
                            'limits'   => ['memory' => '200Mi'],
                            'requests' => ['cpu' => '100m', 'memory' => '200Mi'],
                        ],
                        'volumeMounts' => [
                            ['name' => 'varlog',        'mountPath' => '/var/log'],
                            ['name' => 'varlibdockercontainers', 'mountPath' =>
    '/var/lib/docker/containers', 'readOnly' => true],
                        ],
                    ]],
                    'volumes' => [
                        ['name' => 'varlog', 'hostPath' => ['path' => '/var/log']],
                        ['name' => 'varlibdockercontainers', 'hostPath' => ['path'
    => '/var/lib/docker/containers']],
                    ],
                ],
            ],
        ],
    ]);

    $client->daemonSets()->create($daemonSet);
    echo "Fluentd DaemonSet deployed to all nodes\n";
```

DaemonSet with Node Selector (Specific Nodes Only)

```php
$daemonSet = new DaemonSet([
    'metadata' => ['name' => 'gpu-monitor'],
    'spec' => [
        'selector'  => ['matchLabels' => ['app' => 'gpu-monitor']],
        'template'  => [
            'metadata' => ['labels' => ['app' => 'gpu-monitor']],
            'spec'     => [
                'nodeSelector' => ['gpu' => 'true'],   // Only nodes with this label
                'containers'   => [[
                    'name'  => 'gpu-monitor',
                    'image' => 'gpu-metrics:latest',
                ]],
            ],
        ],
    ],
]);

$client->daemonSets()->create($daemonSet);
```

Check DaemonSet Health

```php
function isDaemonSetHealthy(Client $client, string $name): bool
{
    $ds = $client->daemonSets()->findByName($name);
    $status = $ds->getStatus();

    $desired   = $status['desiredNumberScheduled'] ?? 0;
    $ready     = $status['numberReady'] ?? 0;

    return $desired > 0 && $ready === $desired;
}
```

Delete StatefulSet / DaemonSet

```php
// Delete StatefulSet (does NOT delete associated PVCs — they persist)
$client->statefulSets()->deleteByName('mysql');

// Delete PVCs manually after
$pvcs = $client->persistentVolumeClaims()->setLabelSelector(['app' =>
```

```
    'mysql'])->find();
foreach ($pvcs as $pvc) {
    $client->persistentVolumeClaims()->deleteByName($pvc->getMetadata('name'));
}

// Delete DaemonSet
$client->daemonSets()->deleteByName('fluentd');
```

Chapter 12: Jobs & CronJobs

Jobs vs CronJobs vs Deployments

Type	Purpose	Restarts on Completion
`Deployment`	Long-running service	Yes (Always)
`Job`	One-off task to completion	Only on failure
`CronJob`	Scheduled recurring task	On schedule

Working with Jobs

List Jobs

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

$jobs = $client->jobs()->find();

foreach ($jobs as $job) {
    $name      = $job->getMetadata('name');
    $active    = $job->getStatus()['active'] ?? 0;
    $succeeded = $job->getStatus()['succeeded'] ?? 0;
    $failed    = $job->getStatus()['failed'] ?? 0;
    $startTime = $job->getStatus()['startTime'] ?? '-';

    printf("%-30s active=%-3d succeeded=%-3d failed=%-3d started=%s\n",
```

```
        $name, $active, $succeeded, $failed, $startTime);
}
```

Create a Job

```php
use Maclof\Kubernetes\Models\Job;

$job = new Job([
    'metadata' => [
        'name'      => 'db-migration',
        'namespace' => 'default',
        'labels'    => ['type' => 'migration'],
    ],
    'spec' => [
        'completions'  => 1,         // number of successful pod completions
needed
        'parallelism'  => 1,         // max pods running in parallel
        'backoffLimit' => 3,         // retry on failure (max 3 times)
        'ttlSecondsAfterFinished' => 3600,  // auto-delete 1 hour after
completion
        'template' => [
            'metadata' => ['labels' => ['job' => 'db-migration']],
            'spec' => [
                'restartPolicy' => 'OnFailure',  // or 'Never'
                'containers' => [[
                    'name'    => 'migrator',
                    'image'   => 'myapp:latest',
                    'command' => ['php', 'artisan', 'migrate', '--force'],
                    'env' => [
                        ['name' => 'DB_HOST', 'value' => 'postgres'],
                        ['name' => 'DB_PASSWORD', 'valueFrom' => [
                            'secretKeyRef' => ['name' => 'db-secret', 'key' =>
'password'],
                        ]],
                    ],
                    'resources' => [
                        'requests' => ['memory' => '128Mi', 'cpu' => '100m'],
                        'limits'   => ['memory' => '256Mi', 'cpu' => '500m'],
                    ],
                ]],
            ],
        ],
    ],
]);
```

```
$created = $client->jobs()->create($job);
echo "Job created: " . $created->getMetadata('name') . "\n";
```

Wait for Job Completion

```php
function waitForJob(Client $client, string $jobName, int $timeout = 300):
string
{
    $start = time();
    while (time() - $start < $timeout) {
        $job       = $client->jobs()->findByName($jobName);
        $status    = $job->getStatus();
        $succeeded = $status['succeeded'] ?? 0;
        $failed    = $status['failed'] ?? 0;
        $active    = $status['active'] ?? 0;

        if ($succeeded > 0) {
            echo "Job '$jobName' completed successfully\n";
            return 'succeeded';
        }

        $backoffLimit = $job->getSpec()['backoffLimit'] ?? 3;
        if ($failed > $backoffLimit) {
            echo "Job '$jobName' failed after $failed attempts\n";
            return 'failed';
        }

        echo "Job status: active=$active succeeded=$succeeded
failed=$failed\n";
        sleep(5);
    }

    return 'timeout';
}

$client->jobs()->create($job);
$result = waitForJob($client, 'db-migration');

if ($result !== 'succeeded') {
    // Fetch logs from the failed pod
    $pods = $client->pods()->setLabelSelector(['job' => 'db-migration'])-
>find();
    foreach ($pods as $pod) {
        echo "=== Logs for " . $pod->getMetadata('name') . " ===\n";
        echo $client->pods()->logs($pod->getMetadata('name'), ['previous' =>
```

```
    false]);
        }
    }
```

Parallel Job (Batch Processing)

```php
$job = new Job([
    'metadata' => ['name' => 'batch-processor'],
    'spec' => [
        'completions' => 10,    // process 10 items total
        'parallelism' => 3,     // 3 workers at a time
        'backoffLimit' => 2,
        'template' => [
            'spec' => [
                'restartPolicy' => 'Never',
                'containers' => [[
                    'name'    => 'worker',
                    'image'   => 'myapp:batch',
                    'command' => ['php', 'batch.php'],
                    'env'     => [
                        ['name' => 'JOB_INDEX', 'valueFrom' => [
                            'fieldRef' => ['fieldPath' =>
'metadata.annotations[\'batch.kubernetes.io/job-completion-index\']'],
                        ]],
                    ],
                ]],
            ],
        ],
    ],
]);

$client->jobs()->create($job);
```

Delete a Job

```php
// Delete job and its pods
$client->jobs()->deleteByName('db-migration', [
    'propagationPolicy' => 'Foreground',
]);
```

Working with CronJobs

List CronJobs

```php
$cronJobs = $client->cronJobs()->find();

foreach ($cronJobs as $cj) {
    $name          = $cj->getMetadata('name');
    $schedule      = $cj->getSpec()['schedule'];
    $lastSchedule  = $cj->getStatus()['lastScheduleTime'] ?? 'never';
    $active        = count($cj->getStatus()['active'] ?? []);
    $suspended     = $cj->getSpec()['suspend'] ? 'SUSPENDED' : 'active';

    printf("%-30s %-15s %-20s active=%-2d %s\n",
        $name, $schedule, $lastSchedule, $active, $suspended);
}
```

Create a CronJob

```php
use Maclof\Kubernetes\Models\CronJob;

$cronJob = new CronJob([
    'metadata' => [
        'name'      => 'daily-report',
        'namespace' => 'default',
    ],
    'spec' => [
        'schedule'                  => '0 6 * * *',   // every day at 6am UTC
        'timezone'                  => 'America/New_York',
        'concurrencyPolicy'         => 'Forbid',       // don't run if
previous still running
        'successfulJobsHistoryLimit' => 3,
        'failedJobsHistoryLimit'     => 1,
        'suspend'                   => false,
        'startingDeadlineSeconds'    => 300,           // skip if > 5 min late
        'jobTemplate' => [
            'spec' => [
                'backoffLimit' => 2,
                'ttlSecondsAfterFinished' => 86400,
                'template' => [
                    'spec' => [
                        'restartPolicy' => 'OnFailure',
                        'containers' => [[
```

```
                            'name'    => 'reporter',
                            'image'   => 'myapp:latest',
                            'command' => ['php', 'artisan', 'report:daily'],
                            'resources' => [
                                'requests' => ['memory' => '256Mi', 'cpu' =>
'200m'],
                                'limits'   => ['memory' => '512Mi', 'cpu' =>
'1'],
                            ],
                        ]],
                    ],
                ],
            ],
        ],
    ],
]);

$client->cronJobs()->create($cronJob);
echo "CronJob 'daily-report' scheduled: 0 6 * * *\n";
```

CronJob Schedule Reference

```
# ┌──────── minute (0-59)
# │ ┌────── hour (0-23)
# │ │ ┌──── day of month (1-31)
# │ │ │ ┌ month (1-12)
# │ │ │ │ ┌ day of week (0-6, Sun=0)
# │ │ │ │ │
  * * * * *

Examples:
  0 * * * *     = every hour
  0 6 * * *     = every day at 6am
  */15 * * * *  = every 15 minutes
  0 6 * * 1     = every Monday at 6am
  0 0 1 * *     = first of every month at midnight
```

Suspend / Resume a CronJob

php

```
$cj = $client->cronJobs()->findByName('daily-report');

// Suspend
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => true]]);
```

```
echo "CronJob suspended\n";

// Resume
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => false]]);
echo "CronJob resumed\n";
```

Manually Trigger a CronJob

```php
function triggerCronJob(Client $client, string $cronJobName): string
{
    $cj = $client->cronJobs()->findByName($cronJobName);

    $jobSpec     = $cj->getSpec()['jobTemplate']['spec'];
    $manualJobName = $cronJobName . '-manual-' . time();

    $job = new Job([
        'metadata' => [
            'name'            => $manualJobName,
            'namespace'       => 'default',
            'ownerReferences' => [],
            'annotations'     => ['triggered-by' => 'manual'],
        ],
        'spec' => $jobSpec,
    ]);

    $client->jobs()->create($job);
    echo "Manual trigger: $manualJobName\n";

    return $manualJobName;
}

$jobName = triggerCronJob($client, 'daily-report');
$result  = waitForJob($client, $jobName);
echo "Manual run result: $result\n";
```

List Recent Job Runs from a CronJob

```php
$cj = $client->cronJobs()->findByName('daily-report');

// Active jobs
$activeJobs = $cj->getStatus()['active'] ?? [];
echo "Currently running: " . count($activeJobs) . "\n";
```

```php
// Find historical jobs by label
$historicalJobs = $client->jobs()->setLabelSelector([
    'app' => 'daily-report',
])->find();

foreach ($historicalJobs as $job) {
    $name      = $job->getMetadata('name');
    $startTime = $job->getStatus()['startTime'] ?? '-';
    $endTime   = $job->getStatus()['completionTime'] ?? 'running';
    $succeeded = $job->getStatus()['succeeded'] ?? 0;

    echo "$name  start=$startTime  end=$endTime  success=$succeeded\n";
}
```

Concurrency Policy Options

Policy	Behavior
`Allow` (default)	Multiple instances can run simultaneously
`Forbid`	Skip if previous run still running
`Replace`	Cancel previous run, start new one

Chapter 13: Watching Events & Real-Time Updates

The Watch API

The Kubernetes Watch API streams resource change events in real time over a long-lived HTTP connection. Each event has a type:

Event Type	Meaning
`ADDED`	Resource was created
`MODIFIED`	Resource was updated
`DELETED`	Resource was deleted

Event Type	Meaning
`ERROR`	Watch error (restart the watch)

Watching Pods

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

// Watch all pods in the current namespace
$client->pods()->watch(function (string $type, $pod) {
    $name  = $pod->getMetadata('name');
    $phase = $pod->getStatus()['phase'] ?? 'Unknown';

    switch ($type) {
        case 'ADDED':
            echo "[+] Pod added:    $name ($phase)\n";
            break;
        case 'MODIFIED':
            echo "[~] Pod changed:  $name ($phase)\n";
            break;
        case 'DELETED':
            echo "[-] Pod deleted:  $name\n";
            break;
        case 'ERROR':
            echo "[!] Watch error for $name\n";
            break;
    }

    return true;  // return false to stop the watch
});
```

Watching Pods with Label Filter

```php
$client->pods()->setLabelSelector(['app' => 'my-app'])->watch(function ($type,
$pod) {
```

```
    $name   = $pod->getMetadata('name');
    $reason = $pod->getStatus()['conditions'][0]['reason'] ?? '-';
    echo "[$type] $name — $reason\n";
    return true;
});
```

Watching Deployments (Detect Rollouts)

```php
$client->deployments()->watch(function ($type, $deployment) {
    if ($type !== 'MODIFIED') return true;

    $name      = $deployment->getMetadata('name');
    $desired   = $deployment->getSpec()['replicas'] ?? 0;
    $ready     = $deployment->getStatus()['readyReplicas'] ?? 0;
    $updated   = $deployment->getStatus()['updatedReplicas'] ?? 0;

    echo "Deployment '$name': updated=$updated ready=$ready
desired=$desired\n";

    if ($updated === $desired && $ready === $desired) {
        echo "✓ Deployment '$name' rollout complete!\n";
    }

    return true;
});
```

Watching Events (kubectl get events equivalent)

```php
use Maclof\Kubernetes\Models\Event;

$client->events()->watch(function ($type, $event) {
    if ($type !== 'ADDED') return true;

    $name      = $event->getMetadata('name');
    $reason    = $event->getAttribute('reason') ?? '-';
    $message   = $event->getAttribute('message') ?? '-';
    $objName   = $event->getAttribute('involvedObject')['name'] ?? '-';
```

```
    $objKind   = $event->getAttribute('involvedObject')['kind'] ?? '-';
    $eventType = $event->getAttribute('type') ?? 'Normal';  // Normal or
Warning

    $prefix = $eventType === 'Warning' ? '[WARN]' : '[INFO]';
    echo "$prefix $objKind/$objName — $reason: $message\n";

    return true;
});
```

Filter Warning Events Only

```php
$client->events()->watch(function ($type, $event) {
    $evtType = $event->getAttribute('type') ?? 'Normal';

    if ($evtType !== 'Warning') return true;  // skip non-warnings

    $reason  = $event->getAttribute('reason') ?? '-';
    $message = $event->getAttribute('message') ?? '-';
    $obj     = $event->getAttribute('involvedObject')['name'] ?? '-';
    $count   = $event->getAttribute('count') ?? 1;

    echo "[WARNING x$count] $obj — $reason: $message\n";

    return true;
});
```

Detecting Pod Restarts

```php
function watchForPodRestarts(Client $client, string $namespace = 'default'):
void
{
    $client->setNamespace($namespace);

    $knownRestarts = [];

    $client->pods()->watch(function ($type, $pod) use (&$knownRestarts) {
        if ($type === 'DELETED') {
            unset($knownRestarts[$pod->getMetadata('name')]);
            return true;
```

```
        }

        $name     = $pod->getMetadata('name');
        $statuses = $pod->getStatus()['containerStatuses'] ?? [];

        foreach ($statuses as $cs) {
            $container     = $cs['name'];
            $restartCount  = $cs['restartCount'] ?? 0;
            $key           = "$name/$container";

            $prev = $knownRestarts[$key] ?? 0;

            if ($restartCount > $prev) {
                $diff = $restartCount - $prev;
                echo "[RESTART] Pod=$name Container=$container
restarts=$restartCount (+$diff)\n";

                // Optionally fetch logs
                if ($restartCount > 5) {
                    echo "[CRITICAL] High restart count! Fetching previous
logs...\n";
                    // $logs = $client->pods()->logs($name, ['previous' =>
true, 'container' => $container]);
                }
            }

            $knownRestarts[$key] = $restartCount;
        }

        return true;
    });
}

watchForPodRestarts($client, 'production');
```

Watch with Timeout (Non-blocking)

php

```
function watchWithTimeout(Client $client, int $seconds): void
{
    $deadline = time() + $seconds;

    $client->pods()->watch(function ($type, $pod) use ($deadline) {
```

```
        $name = $pod->getMetadata('name');
        echo "[$type] $name\n";

        // Stop watching after timeout
        return time() < $deadline;
    });

    echo "Watch ended after $seconds seconds\n";
}

watchWithTimeout($client, 30);
```

Polling (Alternative to Watching)

For PHP-FPM environments where long-lived connections aren't suitable, use polling:

```php
function pollDeploymentStatus(Client $client, string $name, int $pollEvery = 5): void
{
    echo "Polling deployment '$name' every {$pollEvery}s...\n";

    while (true) {
        $dep    = $client->deployments()->findByName($name);
        $status = $dep->getStatus();

        $desired   = $dep->getSpec()['replicas'] ?? 0;
        $ready     = $status['readyReplicas'] ?? 0;
        $updated   = $status['updatedReplicas'] ?? 0;

        printf("[%s] desired=%d updated=%d ready=%d\n",
            date('H:i:s'), $desired, $updated, $ready);

        if ($updated === $desired && $ready === $desired) {
            echo "Rollout complete!\n";
            break;
        }

        sleep($pollEvery);
    }
}

pollDeploymentStatus($client, 'my-app');
```

Watching Multiple Resource Types

```php
function startClusterMonitor(Client $client): void
{
    // Note: PHP is single-threaded — use separate processes or queues for parallel watching
    // This example watches pods sequentially for demonstration

    $resources = ['pods', 'deployments', 'services'];

    foreach ($resources as $resource) {
        echo "Starting watch for: $resource\n";
        $client->$resource()->watch(function ($type, $obj) use ($resource) {
            echo "[$resource] $type: " . $obj->getMetadata('name') . "\n";
            return true;
        });
    }
}
```

Resource Version and Watch Resumption

The Watch API uses `resourceVersion` to resume from where it left off:

```php
// Get current resource version from a list
$pods = $client->pods()->find();
$resourceVersion = $pods->getResourceVersion();  // e.g. "12345"

// Watch only events that happened AFTER this version
$client->pods()->watch(function ($type, $pod) {
    echo "[$type] " . $pod->getMetadata('name') . "\n";
    return true;
}, ['resourceVersion' => $resourceVersion]);
```

Kubernetes Events vs Watch Events

	`$client->events()->watch()`	`$client->pods()->watch()`
What it shows	Human-readable cluster events (like `kubectl get events`)	Raw API resource change stream
CrashLoopBackOff	`BackOff` Warning event	MODIFIED pod with `CrashLoopBackOff` status
Scaling event	`ScalingReplicaSet` event	ADDED/DELETED pod events
Best for	Alerting, audit trail	Automation, operators

Chapter 14: Custom Resources (CRDs)

What Are CRDs?

CustomResourceDefinitions (CRDs) let you extend the Kubernetes API with your own resource types. Operators (like cert-manager, ArgoCD, Prometheus Operator) define CRDs to manage their own concepts.

```
Standard K8s API:  Pod, Deployment, Service
CRD examples:      Certificate, Application, PrometheusRule, IngressRoute
```

Listing CRDs in the Cluster

```php
use Maclof\Kubernetes\Client;

$client = Client::fromKubeConfig();

// CRDs are cluster-scoped (not namespaced)
$crds = $client->customResourceDefinitions()->find();

foreach ($crds as $crd) {
    $name    = $crd->getMetadata('name');    // e.g. certificates.cert-manager.io
    $group   = $crd->getSpec()['group'];     // e.g. cert-manager.io
    $kinds   = array_column($crd->getSpec()['names'] ?? [], 'kind');
    $scope   = $crd->getSpec()['scope'];     // Namespaced or Cluster
```

```
    printf("%-55s %-30s %-12s\n", $name, $group, $scope);
}
```

Reading Custom Resources via Raw HTTP

The `maclof/kubernetes-client` library handles standard resources natively. For CRDs, use the raw API or the `customResources` method:

```php
// Generic custom resource access
// Format: /apis/{group}/{version}/namespaces/{namespace}/{plural}

$response = $client->sendRequest(
    'GET',
    '/apis/cert-manager.io/v1/namespaces/default/certificates'
);

$body  = json_decode($response->getBody()->getContents(), true);
$certs = $body['items'] ?? [];

foreach ($certs as $cert) {
    $name   = $cert['metadata']['name'];
    $ready  = collect($cert['status']['conditions'] ?? [])
        ->firstWhere('type', 'Ready')['status'] ?? 'Unknown';
    $expiry = $cert['status']['notAfter'] ?? 'N/A';

    echo "$name — ready=$ready expires=$expiry\n";
}
```

Using php-k8s for CRD Support

The `renoki-co/php-k8s` library has better CRD support via its `K8sResource` mechanism:

```php
use RenokiCo\PhpK8s\KubernetesCluster;
use RenokiCo\PhpK8s\Kinds\K8sResource;

$cluster = KubernetesCluster::fromKubeConfigFile('~/.kube/config');

// Define a custom resource
```

```
class K8sCertificate extends K8sResource
{
    protected static $kind       = 'Certificate';
    protected static $apiVersion = 'cert-manager.io/v1';
    protected static $pluralName = 'certificates';
}

// List all certificates
$certs = K8sCertificate::fromCluster($cluster)->all();

foreach ($certs as $cert) {
    echo $cert->getName() . "\n";
}
```

Creating a Custom Resource

```php
// Create a cert-manager Certificate via raw HTTP
$certBody = [
    'apiVersion' => 'cert-manager.io/v1',
    'kind'       => 'Certificate',
    'metadata'   => [
        'name'      => 'myapp-tls',
        'namespace' => 'default',
    ],
    'spec' => [
        'secretName' => 'myapp-tls-secret',
        'issuerRef'  => [
            'name' => 'letsencrypt-prod',
            'kind' => 'ClusterIssuer',
        ],
        'dnsNames' => [
            'myapp.example.com',
            'www.myapp.example.com',
        ],
        'duration'    => '2160h',   // 90 days
        'renewBefore' => '360h',    // renew 15 days before expiry
    ],
];

$client->sendRequest(
    'POST',
    '/apis/cert-manager.io/v1/namespaces/default/certificates',
```

```
        ['json' => $certBody]
    );

    echo "Certificate resource created\n";
```

Working with ArgoCD Applications

```php
function createArgoCDApp(Client $client, string $name, string $repoUrl, string $path): void
{
    $app = [
        'apiVersion' => 'argoproj.io/v1alpha1',
        'kind'       => 'Application',
        'metadata'   => [
            'name'      => $name,
            'namespace' => 'argocd',
            'finalizers' => ['resources-finalizer.argocd.argoproj.io'],
        ],
        'spec' => [
            'project' => 'default',
            'source'  => [
                'repoURL'        => $repoUrl,
                'targetRevision' => 'HEAD',
                'path'           => $path,
            ],
            'destination' => [
                'server'    => 'https://kubernetes.default.svc',
                'namespace' => 'default',
            ],
            'syncPolicy' => [
                'automated' => [
                    'prune'    => true,
                    'selfHeal' => true,
                ],
                'syncOptions' => ['CreateNamespace=true'],
            ],
        ],
    ];

    $client->sendRequest(
        'POST',
        '/apis/argoproj.io/v1alpha1/namespaces/argocd/applications',
```

```
        ['json' => $app]
    );

    echo "ArgoCD Application '$name' created\n";
}
```

Watching Custom Resources

```php
// Watch cert-manager certificates for status changes
$response = $client->sendRequest(
    'GET',
    '/apis/cert-manager.io/v1/namespaces/default/certificates',
    ['query' => ['watch' => 'true', 'timeoutSeconds' => '60']]
);

$body = $response->getBody();
while (!$body->eof()) {
    $line = $body->read(4096);
    if (empty(trim($line))) continue;

    $event = json_decode($line, true);
    $type  = $event['type'] ?? 'UNKNOWN';
    $name  = $event['object']['metadata']['name'] ?? '-';
    $ready = 'Unknown';

    foreach ($event['object']['status']['conditions'] ?? [] as $cond) {
        if ($cond['type'] === 'Ready') {
            $ready = $cond['status'];
            break;
        }
    }

    echo "[$type] Certificate: $name — Ready: $ready\n";
}
```

Defining Your Own CRD via PHP

php

```
function createCRD(Client $client, string $group, string $plural, string
$kind): void
{
    $crd = [
        'apiVersion' => 'apiextensions.k8s.io/v1',
        'kind'       => 'CustomResourceDefinition',
        'metadata'   => ['name' => "$plural.$group"],
        'spec'       => [
            'group' => $group,
            'scope' => 'Namespaced',
            'names' => [
                'plural'   => $plural,
                'singular' => strtolower($kind),
                'kind'     => $kind,
            ],
            'versions' => [[
                'name'    => 'v1',
                'served'  => true,
                'storage' => true,
                'schema'  => [
                    'openAPIV3Schema' => [
                        'type'       => 'object',
                        'properties' => [
                            'spec' => [
                                'type'       => 'object',
                                'properties' => [
                                    'replicas' => ['type' => 'integer'],
                                    'image'    => ['type' => 'string'],
                                ],
                            ],
                        ],
                    ],
                ],
            ]],
        ],
    ];

    $client->sendRequest(
        'POST',
        '/apis/apiextensions.k8s.io/v1/customresourcedefinitions',
        ['json' => $crd]
    );

    echo "CRD '$plural.$group' created\n";
}
```

```
createCRD($client, 'mycompany.io', 'myapps', 'MyApp');
```

List All Custom Resources of a Type

```php
function listCustomResources(Client $client, string $group, string $version,
string $plural, string $namespace = 'default'): array
{
    $path = "/apis/$group/$version/namespaces/$namespace/$plural";

    $response = $client->sendRequest('GET', $path);
    $body     = json_decode($response->getBody()->getContents(), true);

    return $body['items'] ?? [];
}

$myApps = listCustomResources($client, 'mycompany.io', 'v1', 'myapps');
foreach ($myApps as $app) {
    echo $app['metadata']['name'] . "\n";
}
```

CRD Reference Table

CRD (name.group)	Kind	Used By
`certificates.cert-manager.io`	Certificate	cert-manager
`clusterissuers.cert-manager.io`	ClusterIssuer	cert-manager
`applications.argoproj.io`	Application	ArgoCD
`prometheusrules.monitoring.coreos.com`	PrometheusRule	Prometheus Operator
`ingressroutes.traefik.containo.us`	IngressRoute	Traefik
`helmreleases.helm.toolkit.fluxcd.io`	HelmRelease	FluxCD

Chapter 15: Error Handling & Retry Logic

Kubernetes API Error Codes

HTTP Status	Meaning	Common Cause
`400 Bad Request`	Invalid resource spec	Malformed YAML/JSON
`401 Unauthorized`	Not authenticated	Expired/missing token
`403 Forbidden`	No permission	RBAC not configured
`404 Not Found`	Resource doesn't exist	Wrong name or namespace
`409 Conflict`	Resource already exists	Creating duplicate
`422 Unprocessable`	Validation failed	Invalid field values
`429 Too Many Requests`	Rate limited	Too many API calls
`500 Internal Server Error`	K8s API bug	Rare
`503 Service Unavailable`	API server overloaded	High load

Basic Exception Handling

```php
use Maclof\Kubernetes\Client;
use GuzzleHttp\Exception\ClientException;
use GuzzleHttp\Exception\ConnectException;
use GuzzleHttp\Exception\ServerException;

$client = Client::fromKubeConfig();

function createDeploymentSafe(Client $client, array $spec): bool
{
    try {
        $deployment = new \Maclof\Kubernetes\Models\Deployment($spec);
        $client->deployments()->create($deployment);
        return true;
```

```
    } catch (ClientException $e) {
        $code    = $e->getResponse()->getStatusCode();
        $body    = json_decode($e->getResponse()->getBody()->getContents(),
true);
        $message = $body['message'] ?? $e->getMessage();

        switch ($code) {
            case 401:
                error_log("Authentication failed. Check your
token/certificate.");
                break;
            case 403:
                error_log("Permission denied. Check RBAC for this
ServiceAccount.");
                break;
            case 409:
                error_log("Resource already exists: " . ($body['details']
['name'] ?? 'unknown'));
                break;
            case 422:
                error_log("Validation failed: $message");
                break;
            default:
                error_log("API error $code: $message");
        }
        return false;

    } catch (ConnectException $e) {
        error_log("Cannot connect to Kubernetes API: " . $e->getMessage());
        return false;

    } catch (ServerException $e) {
        error_log("Kubernetes API server error: " . $e->getMessage());
        return false;
    }
}
```

Create-or-Update (Apply) Pattern

```php
function applyDeployment(Client $client, array $spec): string
{
    $name = $spec['metadata']['name'];
```

```
    try {
        $existing = $client->deployments()->findByName($name);

        if ($existing === null) {
            // Create
            $client->deployments()->create(new
\Maclof\Kubernetes\Models\Deployment($spec));
            return 'created';
        } else {
            // Update
            $client->deployments()->patch($existing, $spec);
            return 'updated';
        }

    } catch (ClientException $e) {
        if ($e->getResponse()->getStatusCode() === 404) {
            $client->deployments()->create(new
\Maclof\Kubernetes\Models\Deployment($spec));
            return 'created';
        }
        throw $e;
    }
}

$action = applyDeployment($client, $myDeploymentSpec);
echo "Deployment $action\n";
```

Retry Logic with Exponential Backoff

```php
function withRetry(callable $operation, int $maxAttempts = 3, float $baseDelay
= 1.0): mixed
{
    $attempt = 0;

    while (true) {
        $attempt++;
        try {
            return $operation();

        } catch (ServerException $e) {
            // 5xx errors are transient — retry
```

```
            if ($attempt >= $maxAttempts) throw $e;

            $delay = $baseDelay * (2 ** ($attempt - 1));  // 1s, 2s, 4s
            error_log("Server error (attempt $attempt/$maxAttempts), retrying
in {$delay}s...");
            sleep((int) $delay);

        } catch (ClientException $e) {
            $code = $e->getResponse()->getStatusCode();

            if ($code === 429) {
                // Rate limited — respect Retry-After header
                $retryAfter = (int) ($e->getResponse()->getHeader('Retry-
After')[0] ?? 5);
                error_log("Rate limited, waiting {$retryAfter}s...");
                sleep($retryAfter);
                continue;
            }

            // 4xx errors (except 429) are not retryable
            throw $e;

        } catch (ConnectException $e) {
            if ($attempt >= $maxAttempts) throw $e;
            $delay = $baseDelay * (2 ** ($attempt - 1));
            error_log("Connection failed (attempt $attempt), retrying in
{$delay}s...");
            sleep((int) $delay);
        }
    }
}

// Usage
$pods = withRetry(fn() => $client->pods()->find(), maxAttempts: 3);
```

Idempotent Resource Management

php

```
class K8sResourceManager
{
    public function __construct(private Client $client) {}

    public function ensureNamespace(string $name): void
```

```
    {
        try {
            $this->client->namespaces()->findByName($name);
        } catch (\Exception $e) {
            $this->client->namespaces()->create(
                new \Maclof\Kubernetes\Models\NamespaceModel([
                    'metadata' => ['name' => $name],
                ])
            );
        }
    }

    public function ensureConfigMap(string $name, string $namespace, array
$data): void
    {
        $this->client->setNamespace($namespace);

        try {
            $cm = $this->client->configMaps()->findByName($name);
            if ($cm !== null) {
                $cm->setAttribute('data', array_map('strval', $data));
                $this->client->configMaps()->update($cm);
                return;
            }
        } catch (\Exception $e) {
            // Not found — fall through to create
        }

        $this->client->configMaps()->create(
            new \Maclof\Kubernetes\Models\ConfigMap([
                'metadata' => ['name' => $name, 'namespace' => $namespace],
                'data'     => array_map('strval', $data),
            ])
        );
    }

    public function deleteIfExists(string $resourceType, string $name): bool
    {
        try {
            $this->client->$resourceType()->deleteByName($name);
            return true;
        } catch (ClientException $e) {
            if ($e->getResponse()->getStatusCode() === 404) {
                return false;  // already gone
            }
            throw $e;
```

```
        }
    }
}
```

Validation Before Creating Resources

```php
function validateDeploymentSpec(array $spec): array
{
    $errors = [];

    if (empty($spec['metadata']['name'])) {
        $errors[] = 'metadata.name is required';
    }

    $replicas = $spec['spec']['replicas'] ?? null;
    if ($replicas !== null && (!is_int($replicas) || $replicas < 0)) {
        $errors[] = 'spec.replicas must be a non-negative integer';
    }

    $containers = $spec['spec']['template']['spec']['containers'] ?? [];
    if (empty($containers)) {
        $errors[] = 'spec.template.spec.containers must not be empty';
    }

    foreach ($containers as $i => $c) {
        if (empty($c['name'])) $errors[] = "containers[$i].name is required";
        if (empty($c['image'])) $errors[] = "containers[$i].image is required";
    }

    return $errors;
}

$errors = validateDeploymentSpec($mySpec);
if (!empty($errors)) {
    foreach ($errors as $err) {
        echo "Validation error: $err\n";
    }
    exit(1);
}

$client->deployments()->create(new Deployment($mySpec));
```

Structured Error Logging

```php
function logK8sError(string $operation, string $resource, \Exception $e): void
{
    $context = [
        'operation'  => $operation,
        'resource'   => $resource,
        'error_type' => get_class($e),
        'message'    => $e->getMessage(),
    ];

    if ($e instanceof ClientException) {
        $context['http_status'] = $e->getResponse()->getStatusCode();
        $body = json_decode($e->getResponse()->getBody()->getContents(), true);
        $context['k8s_reason']  = $body['reason'] ?? null;
        $context['k8s_message'] = $body['message'] ?? null;
    }

    error_log(json_encode($context));
}

try {
    $client->deployments()->create($deployment);
} catch (\Exception $e) {
    logK8sError('create', 'Deployment/my-app', $e);
}
```

Timeout Configuration

```php
$client = new Client([
    'master'  => 'https://k8s.example.com:6443',
    'token'   => '...',
    'ca_cert' => '...',
    'timeout' => 30,    // 30 second timeout per request
]);

// For long-running operations (watch, exec), increase timeout
$client = new Client([
    // ...
```

```
    'timeout'         => 0,      // no timeout for watches
    'connect_timeout' => 10,     // but still timeout on initial connect
]);
```

Common Error Patterns and Fixes

```php
// Pattern: Resource already exists (409)
try {
    $client->secrets()->create($secret);
} catch (ClientException $e) {
    if ($e->getResponse()->getStatusCode() === 409) {
        // Update instead
        $existing = $client->secrets()->findByName($secret->getMetadata('name'));
        $client->secrets()->update($existing);
    }
}

// Pattern: Namespace not found
try {
    $client->setNamespace('team-xyz');
    $pods = $client->pods()->find();
} catch (ClientException $e) {
    if ($e->getResponse()->getStatusCode() === 404) {
        echo "Namespace 'team-xyz' does not exist\n";
    }
}

// Pattern: Forbidden — wrong ServiceAccount permissions
try {
    $client->nodes()->find();  // requires cluster-level access
} catch (ClientException $e) {
    if ($e->getResponse()->getStatusCode() === 403) {
        echo "Missing ClusterRole permission to list nodes\n";
    }
}
```

Chapter 16: Production Patterns

1. Operator Pattern in PHP

A Kubernetes Operator is a controller that watches custom resources and reconciles the actual state toward the desired state. While Go is the standard, PHP can implement basic operators for internal SaaS platforms.

```
Watch CRD events
     |
     ▼
Reconcile loop:
  - Read desired state from CRD
  - Read actual state from cluster
  - Create/update/delete resources to match
     |
     ▼
Repeat forever
```

php

```php
use Maclof\Kubernetes\Client;

class TenantOperator
{
    public function __construct(private Client $client) {}

    public function run(): void
    {
        echo "Tenant operator started\n";

        // Watch Tenant CRDs (assuming CRD is defined)
        while (true) {
            try {
                $this->client->sendRequest('GET',
'/apis/mycompany.io/v1/tenants', [
                    'query' => ['watch' => 'true', 'timeoutSeconds' => '3600'],
                    'stream' => true,
                ])->then(function ($response) {
                    $body = $response->getBody();
                    while (!$body->eof()) {
                        $line  = $body->readline();
                        $event = json_decode($line, true);
                        if ($event) {
                            $this->reconcile($event['type'], $event['object']);
                        }
                    }
                });
            } catch (\Exception $e) {
                error_log("Watch error: " . $e->getMessage());
                sleep(5);  // backoff before reconnecting
```

```
            }
        }
    }

    private function reconcile(string $type, array $tenant): void
    {
        $name  = $tenant['metadata']['name'];
        $ns    = $tenant['spec']['namespace'] ?? $name . '-workspace';
        $owner = $tenant['spec']['owner'] ?? '';

        echo "[$type] Reconciling tenant: $name → namespace: $ns\n";

        if ($type === 'DELETED') {
            $this->deprovisionTenant($ns);
            return;
        }

        $this->provisionTenant($ns, $name, $owner);
    }

    private function provisionTenant(string $ns, string $team, string $owner):
void
    {
        // Ensure namespace exists
        $this->ensureNamespace($ns, $team, $owner);

        // Ensure resource quota
        $this->ensureResourceQuota($ns);

        // Ensure limit range
        $this->ensureLimitRange($ns);

        echo "Tenant '$team' provisioned in namespace '$ns'\n";
    }

    private function deprovisionTenant(string $ns): void
    {
        try {
            $this->client->namespaces()->deleteByName($ns);
            echo "Namespace '$ns' deleted\n";
        } catch (\Exception $e) {
            error_log("Failed to delete namespace '$ns': " . $e->getMessage());
        }
    }

    private function ensureNamespace(string $name, string $team, string
```

```
    $owner): void
        {
            try {
                $this->client->namespaces()->findByName($name);
            } catch (\Exception $e) {
                $this->client->namespaces()->create(new
    \Maclof\Kubernetes\Models\NamespaceModel([
                    'metadata' => [
                        'name'        => $name,
                        'labels'      => ['team' => $team, 'managed-by' => 'tenant-
    operator'],
                        'annotations' => ['owner' => $owner],
                    ],
                ]));
            }
        }

        private function ensureResourceQuota(string $ns): void
        {
            $this->client->setNamespace($ns);

            // Basic quota: 4 CPU, 8Gi RAM, 10 pods
            $quota = [
                'metadata' => ['name' => 'tenant-quota', 'namespace' => $ns],
                'spec' => ['hard' => [
                    'requests.cpu'    => '4',
                    'requests.memory' => '8Gi',
                    'limits.cpu'      => '8',
                    'limits.memory'   => '16Gi',
                    'pods'            => '20',
                ]],
            ];

            try {
                $existing = $this->client->resourceQuotas()->findByName('tenant-
    quota');
                $existing->setAttribute('spec', $quota['spec']);
                $this->client->resourceQuotas()->update($existing);
            } catch (\Exception $e) {
                $this->client->resourceQuotas()->create(
                    new \Maclof\Kubernetes\Models\ResourceQuota($quota)
                );
            }
        }

        private function ensureLimitRange(string $ns): void
```

```
    {
        $this->client->setNamespace($ns);

        $limitRange = [
            'metadata' => ['name' => 'tenant-limits', 'namespace' => $ns],
            'spec' => ['limits' => [[
                'type'           => 'Container',
                'defaultRequest' => ['cpu' => '100m', 'memory' => '128Mi'],
                'default'        => ['cpu' => '500m', 'memory' => '256Mi'],
            ]]],
        ];

        // create or update...
    }
}

// Entry point
$client   = Client::fromKubeConfig();
$operator = new TenantOperator($client);
$operator->run();
```

---

2. Multi-Cluster Management

php

```
class MultiClusterClient
{
    private array $clients = [];

    public function addCluster(string $name, Client $client): void
    {
        $this->clients[$name] = $client;
    }

    public function forCluster(string $name): Client
    {
        return $this->clients[$name] ?? throw new
InvalidArgumentException("Unknown cluster: $name");
    }

    public function allClusters(): array
    {
        return array_keys($this->clients);
    }
```

```
    public function aggregatePods(string $namespace = 'default'): array
    {
        $result = [];
        foreach ($this->clients as $clusterName => $client) {
            $client->setNamespace($namespace);
            foreach ($client->pods()->find() as $pod) {
                $result[] = [
                    'cluster' => $clusterName,
                    'name'    => $pod->getMetadata('name'),
                    'phase'   => $pod->getStatus()['phase'] ?? 'Unknown',
                ];
            }
        }
        return $result;
    }
}

$manager = new MultiClusterClient();
$manager->addCluster('us-east', Client::fromKubeConfig(context: 'eks-us-
east'));
$manager->addCluster('eu-west', Client::fromKubeConfig(context: 'gke-eu-
west'));
$manager->addCluster('ap-south', Client::fromKubeConfig(context: 'aks-ap-
south'));

$allPods = $manager->aggregatePods('production');
echo "Total pods across all clusters: " . count($allPods) . "\n";
```

---

3. Deployment Pipeline Integration

php

```
class KubernetesDeployer
{
    public function __construct(private Client $client) {}

    public function deploy(string $image, string $deploymentName, string
$containerName): array
    {
        $startTime = time();

        // 1. Update image
        $dep = $this->client->deployments()->findByName($deploymentName);
```

```
        $this->client->deployments()->patch($dep, [
            'spec' => ['template' => ['spec' => ['containers' => [
                ['name' => $containerName, 'image' => $image],
            ]]]],
        ]);

        echo "Rolling out $image to $deploymentName...\n";

        // 2. Wait for rollout
        $success = $this->waitForRollout($deploymentName);
        $elapsed = time() - $startTime;

        return [
            'success'  => $success,
            'image'    => $image,
            'duration' => $elapsed,
        ];
    }

    private function waitForRollout(string $name, int $timeout = 300): bool
    {
        $start = time();
        while (time() - $start < $timeout) {
            $dep       = $this->client->deployments()->findByName($name);
            $status    = $dep->getStatus();
            $desired   = $dep->getSpec()['replicas'] ?? 0;
            $updated   = $status['updatedReplicas'] ?? 0;
            $ready     = $status['readyReplicas'] ?? 0;
            $available = $status['availableReplicas'] ?? 0;

            printf("  updated=%-3d ready=%-3d available=%-3d desired=%-3d\n",
                $updated, $ready, $available, $desired);

            if ($updated === $desired && $ready === $desired && $available ===
$desired) {
                return true;
            }

            sleep(3);
        }
        return false;
    }

    public function rollback(string $deploymentName): void
    {
        // Trigger rollback by removing the latest ReplicaSet
```

```
        // In practice: patch with previous image or use revision annotation
        $dep = $this->client->deployments()->findByName($deploymentName);
        $this->client->deployments()->patch($dep, [
            'metadata' => ['annotations' =>
['deployment.kubernetes.io/revision' => null]],
        ]);
        echo "Rollback triggered for $deploymentName\n";
    }
}

// CI/CD usage
$deployer = new KubernetesDeployer($client);
$result   = $deployer->deploy('myapp:2.5.0', 'my-app', 'app');

if (!$result['success']) {
    echo "Deploy failed after {$result['duration']}s — rolling back\n";
    $deployer->rollback('my-app');
    exit(1);
}

echo "Deploy succeeded in {$result['duration']}s\n";
```

---

4. Cluster Health Dashboard

php

```
function clusterHealthReport(Client $client): array
{
    $client->setNamespace('');  // all namespaces

    // Nodes
    $nodes       = $client->nodes()->find();
    $readyNodes  = 0;
    foreach ($nodes as $node) {
        $conditions = $node->getStatus()['conditions'] ?? [];
        foreach ($conditions as $cond) {
            if ($cond['type'] === 'Ready' && $cond['status'] === 'True') {
                $readyNodes++;
                break;
            }
        }
    }

    // Pods
```

```
    $pods        = $client->pods()->find();
    $runningPods = 0;
    $failedPods  = 0;
    foreach ($pods as $pod) {
        $phase = $pod->getStatus()['phase'] ?? 'Unknown';
        if ($phase === 'Running') $runningPods++;
        if ($phase === 'Failed')  $failedPods++;
    }

    // Deployments
    $deployments       = $client->deployments()->find();
    $healthyDeployments = 0;
    foreach ($deployments as $dep) {
        $desired = $dep->getSpec()['replicas'] ?? 0;
        $ready   = $dep->getStatus()['readyReplicas'] ?? 0;
        if ($ready === $desired) $healthyDeployments++;
    }

    return [
        'nodes'        => ['total' => count($nodes), 'ready' => $readyNodes],
        'pods'         => ['total' => count($pods), 'running' => $runningPods,
'failed' => $failedPods],
        'deployments'  => ['total' => count($deployments), 'healthy' =>
$healthyDeployments],
        'timestamp'    => date('c'),
    ];
}

$health = clusterHealthReport($client);
echo json_encode($health, JSON_PRETTY_PRINT);
```

---

5. Canary Deployment Pattern

```
function canaryDeploy(Client $client, string $app, string $newImage, int
$canaryWeight = 10): void
{
    // Scale down canary deployment to ~10% of traffic weight
    // Requires a traffic management solution (Istio, Argo Rollouts, etc.)
    // This example manages it with two Deployments + Services

    $totalReplicas  = 10;
    $canaryReplicas = (int) ceil($totalReplicas * ($canaryWeight / 100));
```

php

```
    $stableReplicas = $totalReplicas - $canaryReplicas;

    // Patch stable deployment
    $stable = $client->deployments()->findByName($app);
    $client->deployments()->patch($stable, ['spec' => ['replicas' =>
$stableReplicas]]);

    // Create or update canary deployment
    $canaryName = $app . '-canary';
    try {
        $canary = $client->deployments()->findByName($canaryName);
        $client->deployments()->patch($canary, [
            'spec' => [
                'replicas' => $canaryReplicas,
                'template' => ['spec' => ['containers' => [['name' => $app,
'image' => $newImage]]]],
            ],
        ]);
    } catch (\Exception $e) {
        // Create canary from stable spec
        $stableSpec         = $stable->getSpec();
        $stableSpec['replicas'] = $canaryReplicas;
        $stableSpec['template']['spec']['containers'][0]['image'] = $newImage;
        $stableSpec['selector']['matchLabels']['track'] = 'canary';
        $stableSpec['template']['metadata']['labels']['track'] = 'canary';

        $client->deployments()->create(new
\Maclof\Kubernetes\Models\Deployment([
            'metadata' => ['name' => $canaryName, 'labels' => ['app' => $app,
'track' => 'canary']],
            'spec'     => $stableSpec,
        ]));
    }

    echo "Canary deployed: $canaryReplicas/{$totalReplicas} pods running
$newImage\n";
}

// Promote canary → full rollout
function promoteCanary(Client $client, string $app, string $newImage): void
{
    $totalReplicas = 10;

    $stable = $client->deployments()->findByName($app);
    $client->deployments()->patch($stable, [
        'spec' => [
```

```
            'replicas' => $totalReplicas,
            'template' => ['spec' => ['containers' => [['name' => $app, 'image'
=> $newImage]]]],
        ],
    ]);

    $client->deployments()->deleteByName($app . '-canary', ['propagationPolicy'
=> 'Foreground']);
    echo "Canary promoted — full rollout complete\n";
}
```

---

6. Automated Cleanup Job

```php
function cleanupCompletedJobs(Client $client, int $keepSuccessful = 3, int
$keepFailed = 1): void
{
    $jobs = $client->jobs()->find();
    $succeeded = $failed = [];

    foreach ($jobs as $job) {
        $status = $job->getStatus();
        if (($status['succeeded'] ?? 0) > 0) {
            $succeeded[] = $job;
        } elseif (($status['failed'] ?? 0) > 0) {
            $failed[] = $job;
        }
    }

    // Sort oldest first
    usort($succeeded, fn($a, $b) =>
        strtotime($a->getStatus()['completionTime'] ?? 0)
        <=> strtotime($b->getStatus()['completionTime'] ?? 0)
    );

    $toDelete = array_slice($succeeded, 0, max(0, count($succeeded) -
$keepSuccessful));
    foreach ($toDelete as $job) {
        $client->jobs()->delete($job, ['propagationPolicy' => 'Foreground']);
        echo "Deleted completed job: " . $job->getMetadata('name') . "\n";
    }

    $toDeleteFailed = array_slice($failed, 0, max(0, count($failed) -
```

```
    $keepFailed));
        foreach ($toDeleteFailed as $job) {
            $client->jobs()->delete($job, ['propagationPolicy' => 'Foreground']);
            echo "Deleted failed job: " . $job->getMetadata('name') . "\n";
        }
    }

    cleanupCompletedJobs($client, keepSuccessful: 5, keepFailed: 2);
```

Production Checklist

Concern	Solution
Authentication	Use in-cluster ServiceAccount tokens, not hardcoded certs
Least privilege	ServiceAccount with only the Roles it needs
Retry logic	Exponential backoff for 5xx and 429 errors
Timeout	Always set connection + request timeouts
Logging	Structured logs with cluster, namespace, resource context
Secret handling	Never log Secret values; use `valueFrom.secretKeyRef` in pod specs
Multi-cluster	Separate Client instances per cluster, aggregate carefully
Rate limiting	Batch API calls; avoid listing all resources on every request
Watch reconnection	Auto-restart watch on EOF or timeout
idempotency	Use create-or-update pattern for all resource management

k8s-client-php — Master Cheatsheet

Install

```
composer require maclof/kubernetes-client
# Laravel alternative:
composer require renoki-co/php-k8s
```

Connect

```php
use Maclof\Kubernetes\Client;

// From kubeconfig (dev)
$client = Client::fromKubeConfig();
$client = Client::fromKubeConfig(context: 'my-cluster');

// In-cluster (Pod inside K8s)
$client = new Client([
    'master'    => 'https://kubernetes.default.svc',
    'ca_cert'   => '/var/run/secrets/kubernetes.io/serviceaccount/ca.crt',
    'token'     =>
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/token'),
    'namespace' =>
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/namespace'),
]);

// Client cert
$client = new Client([
    'master'      => 'https://k8s.example.com:6443',
    'ca_cert'     => '/path/to/ca.crt',
    'client_cert' => '/path/to/client.crt',
    'client_key'  => '/path/to/client.key',
]);

// Bearer token
$client = new Client([
    'master'  => 'https://k8s.example.com:6443',
    'ca_cert' => '/path/to/ca.crt',
    'token'   => 'eyJhbGci...',
]);

// Switch namespace
$client->setNamespace('production');
$client->setNamespace('');  // all namespaces
```

Pods

```php
// List
$client->pods()->find()
$client->pods()->setLabelSelector(['app' => 'nginx'])->find()
$client->pods()->setFieldSelector(['status.phase' => 'Running'])->find()

// Get
$client->pods()->findByName('my-pod')

// Logs
$client->pods()->logs('my-pod')
$client->pods()->logs('my-pod', ['tailLines' => 100])
$client->pods()->logs('my-pod', ['sinceSeconds' => 3600])
$client->pods()->logs('my-pod', ['previous' => true])          // crashed container
$client->pods()->logs('my-pod', ['container' => 'sidecar'])    // multi-container

// Create
$client->pods()->create(new Pod([
    'metadata' => ['name' => 'my-pod', 'labels' => ['app' => 'nginx']],
    'spec'     => ['containers' => [['name' => 'nginx', 'image' => 'nginx:alpine']]],
]))

// Delete
$client->pods()->deleteByName('my-pod')
$client->pods()->delete($pod, ['gracePeriodSeconds' => 0])     // force

// Read pod data
$pod->getMetadata('name')
$pod->getMetadata('namespace')
$pod->getSpec()['nodeName']
$pod->getStatus()['phase']             // Pending | Running | Succeeded | Failed
$pod->getStatus()['podIP']
$pod->getStatus()['containerStatuses']  // array of container states
```

Deployments

```php
// List / Get
$client->deployments()->find()
$client->deployments()->findByName('my-app')

// Create
$client->deployments()->create(new Deployment([...]))

// Scale
$client->deployments()->patch($dep, ['spec' => ['replicas' => 5]])

// Update image (rolling update)
$client->deployments()->patch($dep, [
    'spec' => ['template' => ['spec' => ['containers' => [
        ['name' => 'app', 'image' => 'myapp:2.0'],
    ]]]],
])

// Delete
$client->deployments()->deleteByName('my-app')

// Read status
$dep->getSpec()['replicas']
$dep->getStatus()['readyReplicas']
$dep->getStatus()['updatedReplicas']
$dep->getStatus()['availableReplicas']
```

Services

```php
// List / Get
$client->services()->find()
$client->services()->findByName('my-svc')

// Create ClusterIP
$client->services()->create(new Service([
    'metadata' => ['name' => 'my-svc'],
    'spec'     => [
        'type'     => 'ClusterIP',
        'selector' => ['app' => 'my-app'],
        'ports'    => [['port' => 80, 'targetPort' => 8080]],
```

```
    ],
]))

// Create LoadBalancer
// (same but 'type' => 'LoadBalancer')

// Read
$svc->getSpec()['clusterIP']
$svc->getSpec()['type']
$svc->getStatus()['loadBalancer']['ingress']   // external IPs

// Delete
$client->services()->deleteByName('my-svc')
```

ConfigMaps

```php
// List / Get
$client->configMaps()->find()
$client->configMaps()->findByName('app-config')

// Read values
$cm->getAttribute('data')['MY_KEY']

// Create
$client->configMaps()->create(new ConfigMap([
    'metadata' => ['name' => 'app-config'],
    'data'     => ['KEY' => 'value', 'OTHER' => 'val'],
]))

// Update
$cm = $client->configMaps()->findByName('app-config');
$data = $cm->getAttribute('data');
$data['KEY'] = 'new-value';
$cm->setAttribute('data', $data);
$client->configMaps()->update($cm);

// Delete
$client->configMaps()->deleteByName('app-config')
```

Secrets

```php
// List / Get  (never print values!)
$client->secrets()->find()
$client->secrets()->findByName('db-creds')

// Read (always base64_decode)
$val = base64_decode($secret->getAttribute('data')['password'])

// Create (always base64_encode values)
$client->secrets()->create(new Secret([
    'metadata' => ['name' => 'db-creds'],
    'type'     => 'Opaque',
    'data'     => [
        'username' => base64_encode('admin'),
        'password' => base64_encode('secret'),
    ],
]))

// Docker registry secret
'type' => 'kubernetes.io/dockerconfigjson',
'data' => ['.dockerconfigjson' => base64_encode(json_encode(['auths' =>
[...]]))],

// TLS secret
'type' => 'kubernetes.io/tls',
'data' => ['tls.crt' => base64_encode($cert), 'tls.key' =>
base64_encode($key)],

// Delete
$client->secrets()->deleteByName('db-creds')
```

Namespaces

```php
$client->namespaces()->find()
$client->namespaces()->findByName('production')

$client->namespaces()->create(new NamespaceModel([
    'metadata' => ['name' => 'team-alpha', 'labels' => ['team' => 'alpha']],
]))
```

```
$client->namespaces()->deleteByName('team-alpha')  // deletes ALL resources inside!
```

PVC / Storage

```php
$client->persistentVolumeClaims()->find()
$client->persistentVolumeClaims()->findByName('my-pvc')

$client->persistentVolumeClaims()->create(new PersistentVolumeClaim([
    'metadata' => ['name' => 'my-pvc'],
    'spec'     => [
        'accessModes'      => ['ReadWriteOnce'],
        'storageClassName' => 'standard',
        'resources'        => ['requests' => ['storage' => '10Gi']],
    ],
]))

// Status
$pvc->getStatus()['phase']          // Pending | Bound | Lost
$pvc->getSpec()['volumeName']       // which PV it's bound to

$client->persistentVolumeClaims()->deleteByName('my-pvc')
$client->persistentVolumes()->find()
$client->storageClasses()->find()
```

Jobs & CronJobs

```php
// Jobs
$client->jobs()->find()
$client->jobs()->findByName('my-job')
$client->jobs()->create(new Job([
    'metadata' => ['name' => 'my-job'],
    'spec' => [
        'backoffLimit' => 3,
        'ttlSecondsAfterFinished' => 3600,
        'template' => ['spec' => [
            'restartPolicy' => 'OnFailure',
```

```
            'containers'    => [['name' => 'worker', 'image' => 'myapp:latest',
'command' => ['php', 'task.php']]],
        ]],
    ],
]))
$job->getStatus()['succeeded']
$job->getStatus()['failed']
$job->getStatus()['active']
$client->jobs()->deleteByName('my-job', ['propagationPolicy' => 'Foreground'])

// CronJobs
$client->cronJobs()->find()
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => true]])   // pause
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => false]])  // resume
$cj->getSpec()['schedule']
$cj->getStatus()['lastScheduleTime']
```

StatefulSets & DaemonSets

php

```
$client->statefulSets()->find()
$client->statefulSets()->findByName('mysql')
$client->statefulSets()->patch($sts, ['spec' => ['replicas' => 5]])
$sts->getStatus()['readyReplicas']

$client->daemonSets()->find()
$ds->getStatus()['desiredNumberScheduled']
$ds->getStatus()['numberReady']
```

Ingress

php

```
$client->ingresses()->find()
$client->ingresses()->findByName('my-ingress')
$client->ingresses()->create(new Ingress([
    'metadata' => ['name' => 'my-ingress', 'annotations' => [
        'nginx.ingress.kubernetes.io/rewrite-target' => '/',
    ]],
    'spec' => [
```

```
        'ingressClassName' => 'nginx',
        'rules' => [[
            'host' => 'myapp.example.com',
            'http' => ['paths' => [[
                'path'     => '/',
                'pathType' => 'Prefix',
                'backend'  => ['service' => ['name' => 'my-svc', 'port' =>
['number' => 80]]],
            ]]],
        ]],
        'tls' => [['hosts' => ['myapp.example.com'], 'secretName' => 'my-
tls']],
    ],
]))
$client->ingresses()->deleteByName('my-ingress')
```

RBAC

php

```
// Roles
$client->roles()->create(new Role([
    'metadata' => ['name' => 'pod-reader', 'namespace' => 'default'],
    'rules'    => [['apiGroups' => [''], 'resources' => ['pods'], 'verbs' =>
['get', 'list', 'watch']]],
]))

// ClusterRoles
$client->clusterRoles()->create(new ClusterRole([
    'metadata' => ['name' => 'cluster-reader'],
    'rules'    => [['apiGroups' => ['*'], 'resources' => ['*'], 'verbs' =>
['get', 'list', 'watch']]],
]))

// RoleBindings
$client->roleBindings()->create(new RoleBinding([
    'metadata' => ['name' => 'ci-binding', 'namespace' => 'default'],
    'subjects' => [['kind' => 'ServiceAccount', 'name' => 'ci-sa', 'namespace'
=> 'default']],
    'roleRef'  => ['apiGroup' => 'rbac.authorization.k8s.io', 'kind' => 'Role',
'name' => 'pod-reader'],
]))

// ServiceAccounts
```

```
$client->serviceAccounts()->create(new ServiceAccount(['metadata' => ['name' =>
'ci-sa']]))
$client->serviceAccounts()->find()
```

Watch (Real-time Events)

```php
$client->pods()->watch(function ($type, $pod) {
    // $type = ADDED | MODIFIED | DELETED | ERROR
    echo "[$type] " . $pod->getMetadata('name') . "\n";
    return true;   // return false to stop watching
});

$client->deployments()->watch(function ($type, $dep) {
    return true;
});

$client->events()->watch(function ($type, $event) {
    if ($event->getAttribute('type') === 'Warning') {
        echo $event->getAttribute('message') . "\n";
    }
    return true;
});
```

Nodes

```php
$client->nodes()->find()
$client->nodes()->findByName('node-1')

$node->getStatus()['conditions']        // Ready, MemoryPressure, DiskPressure
$node->getStatus()['capacity']          // cpu, memory, pods
$node->getStatus()['allocatable']       // available capacity
$node->getMetadata('labels')            // node labels
```

Error Handling

```php
use GuzzleHttp\Exception\ClientException;
use GuzzleHttp\Exception\ConnectException;

try {
    $client->pods()->find();
} catch (ClientException $e) {
    $code = $e->getResponse()->getStatusCode();
    // 401 = bad token | 403 = no RBAC | 404 = not found | 409 = conflict | 422 = invalid spec
} catch (ConnectException $e) {
    // API server unreachable
}
```

Model Imports

```php
use Maclof\Kubernetes\Models\Pod;
use Maclof\Kubernetes\Models\Deployment;
use Maclof\Kubernetes\Models\Service;
use Maclof\Kubernetes\Models\ConfigMap;
use Maclof\Kubernetes\Models\Secret;
use Maclof\Kubernetes\Models\PersistentVolume;
use Maclof\Kubernetes\Models\PersistentVolumeClaim;
use Maclof\Kubernetes\Models\StorageClass;
use Maclof\Kubernetes\Models\StatefulSet;
use Maclof\Kubernetes\Models\DaemonSet;
use Maclof\Kubernetes\Models\Job;
use Maclof\Kubernetes\Models\CronJob;
use Maclof\Kubernetes\Models\Ingress;
use Maclof\Kubernetes\Models\NamespaceModel;
use Maclof\Kubernetes\Models\ServiceAccount;
use Maclof\Kubernetes\Models\Role;
use Maclof\Kubernetes\Models\ClusterRole;
use Maclof\Kubernetes\Models\RoleBinding;
use Maclof\Kubernetes\Models\ClusterRoleBinding;
```

k8s-client-php — Code Snippets Cheatsheet

Common copy-paste patterns for real-world tasks.

Apply (Create or Update)

```php
function apply(Client $client, string $resource, string $name, array $spec): string
{
    try {
        $existing = $client->$resource()->findByName($name);
        if ($existing) {
            $client->$resource()->patch($existing, $spec);
            return 'updated';
        }
    } catch (\Exception $e) {}

    $modelClass = getModelClass($resource);
    $client->$resource()->create(new $modelClass($spec));
    return 'created';
}
```

Wait for Pod Running

```php
function waitForPodRunning(Client $client, string $name, int $timeout = 60): bool
{
    $start = time();
    while (time() - $start < $timeout) {
        $pod   = $client->pods()->findByName($name);
        $phase = $pod?->getStatus()['phase'] ?? 'Pending';
        if ($phase === 'Running') return true;
        sleep(2);
    }
    return false;
}
```

Wait for Deployment Ready

```php
function waitForDeployment(Client $client, string $name, int $timeout = 120): bool
{
    $start = time();
    while (time() - $start < $timeout) {
        $dep     = $client->deployments()->findByName($name);
        $desired = $dep->getSpec()['replicas'] ?? 0;
        $ready   = $dep->getStatus()['readyReplicas'] ?? 0;
        $updated = $dep->getStatus()['updatedReplicas'] ?? 0;
        if ($updated === $desired && $ready === $desired) return true;
        sleep(3);
    }
    return false;
}
```

Wait for Job Done

```php
function waitForJob(Client $client, string $name, int $timeout = 300): string
{
    $start = time();
    while (time() - $start < $timeout) {
        $job          = $client->jobs()->findByName($name);
        $backoffLimit = $job->getSpec()['backoffLimit'] ?? 3;
        $succeeded    = $job->getStatus()['succeeded'] ?? 0;
        $failed       = $job->getStatus()['failed'] ?? 0;
        if ($succeeded > 0) return 'succeeded';
        if ($failed > $backoffLimit) return 'failed';
        sleep(5);
    }
    return 'timeout';
}
```

Wait for PVC Bound

```php
function waitForPVC(Client $client, string $name, int $timeout = 60): bool
{
    $start = time();
    while (time() - $start < $timeout) {
        $pvc   = $client->persistentVolumeClaims()->findByName($name);
        $phase = $pvc?->getStatus()['phase'] ?? 'Pending';
        if ($phase === 'Bound') return true;
        sleep(3);
    }
    return false;
}
```

Wait for LoadBalancer IP

```php
function waitForLBIP(Client $client, string $name, int $timeout = 120): ?string
{
    $start = time();
    while (time() - $start < $timeout) {
        $svc     = $client->services()->findByName($name);
        $ingress = $svc->getStatus()['loadBalancer']['ingress'] ?? [];
        if (!empty($ingress)) return $ingress[0]['ip'] ?? $ingress[0]
['hostname'] ?? null;
        sleep(5);
    }
    return null;
}
```

Scale Deployment

```php
function scale(Client $client, string $name, int $replicas): void
{
    $dep = $client->deployments()->findByName($name);
    $client->deployments()->patch($dep, ['spec' => ['replicas' => $replicas]]);
}

scale($client, 'my-app', 0);   // scale to zero
scale($client, 'my-app', 5);   // scale up
```

Update Image (Trigger Rolling Update)

```php
function updateImage(Client $client, string $deployment, string $container,
string $image): void
{
    $dep = $client->deployments()->findByName($deployment);
    $client->deployments()->patch($dep, [
        'spec' => ['template' => ['spec' => ['containers' => [
            ['name' => $container, 'image' => $image],
        ]]]],
    ]);
}

updateImage($client, 'my-app', 'app', 'myregistry/myapp:v2.0.0');
```

Get Pods for a Deployment

```php
function getDeploymentPods(Client $client, string $deploymentName): array
{
    $dep      = $client->deployments()->findByName($deploymentName);
    $labels   = $dep->getSpec()['selector']['matchLabels'] ?? [];
    $pods     = $client->pods()->setLabelSelector($labels)->find();
    return iterator_to_array($pods);
}
```

Get Pod Logs for a Deployment

```php
function getDeploymentLogs(Client $client, string $deploymentName, int
$tailLines = 100): array
{
    $logs = [];
    foreach (getDeploymentPods($client, $deploymentName) as $pod) {
        $name         = $pod->getMetadata('name');
        $logs[$name]  = $client->pods()->logs($name, ['tailLines' =>
$tailLines]);
```

```
    }
    return $logs;
}
```

Restart a Deployment (Rollout Restart)

```php
function restartDeployment(Client $client, string $name): void
{
    $dep = $client->deployments()->findByName($name);
    // Add restart annotation to force pod recreation
    $client->deployments()->patch($dep, [
        'spec' => ['template' => ['metadata' => ['annotations' => [
            'kubectl.kubernetes.io/restartedAt' => date('c'),
        ]]]],
    ]);
}
```

Sync ConfigMap from PHP Array

```php
function syncConfigMap(Client $client, string $name, array $config): void
{
    $data = array_map('strval', $config);
    try {
        $cm = $client->configMaps()->findByName($name);
        $cm->setAttribute('data', $data);
        $client->configMaps()->update($cm);
    } catch (\Exception $e) {
        $client->configMaps()->create(new ConfigMap([
            'metadata' => ['name' => $name],
            'data'     => $data,
        ]));
    }
}

syncConfigMap($client, 'feature-flags', [
    'FEATURE_NEW_UI'    => 'true',
```

```
    'FEATURE_DARK_MODE' => 'false',
]);
```

Rotate a Secret

```php
function rotateSecret(Client $client, string $name, string $key, string
$newValue): void
{
    $secret = $client->secrets()->findByName($name);
    $data   = $secret->getAttribute('data') ?? [];
    $data[$key . '_previous'] = $data[$key] ?? '';
    $data[$key] = base64_encode($newValue);
    $secret->setAttribute('data', $data);
    $client->secrets()->update($secret);
}
```

Provision Tenant Namespace

```php
function provisionTenant(Client $client, string $team): void
{
    $ns = "$team-workspace";

    // Namespace
    $client->namespaces()->create(new NamespaceModel([
        'metadata' => ['name' => $ns, 'labels' => ['team' => $team]],
    ]));

    $client->setNamespace($ns);

    // ServiceAccount
    $client->serviceAccounts()->create(new ServiceAccount([
        'metadata' => ['name' => "$team-sa", 'namespace' => $ns],
    ]));

    // Role (full access in namespace)
    $client->roles()->create(new Role([
        'metadata' => ['name' => "$team-role", 'namespace' => $ns],
```

```
            'rules'     => [['apiGroups' => ['', 'apps', 'batch'], 'resources' =>
['*'], 'verbs' => ['*']]],
        ]));

        // Bind SA to Role
        $client->roleBindings()->create(new RoleBinding([
            'metadata' => ['name' => "$team-binding", 'namespace' => $ns],
            'subjects' => [['kind' => 'ServiceAccount', 'name' => "$team-sa",
'namespace' => $ns]],
            'roleRef'  => ['apiGroup' => 'rbac.authorization.k8s.io', 'kind' =>
'Role', 'name' => "$team-role"],
        ]));
}
```

Cluster-wide Pod Status Summary

```php
function podSummary(Client $client): void
{
    $client->setNamespace('');
    $phases = ['Running' => 0, 'Pending' => 0, 'Failed' => 0, 'Succeeded' => 0,
'Unknown' => 0];
    foreach ($client->pods()->find() as $pod) {
        $phase = $pod->getStatus()['phase'] ?? 'Unknown';
        $phases[$phase] = ($phases[$phase] ?? 0) + 1;
    }
    foreach ($phases as $phase => $count) {
        printf("%-12s %d\n", $phase, $count);
    }
}
```

Delete All Pods with Label (Force Restart)

```php
function forceRestartPods(Client $client, array $labels): void
{
    $pods = $client->pods()->setLabelSelector($labels)->find();
    foreach ($pods as $pod) {
        $name = $pod->getMetadata('name');
```

```
        $client->pods()->delete($pod, ['gracePeriodSeconds' => 0]);
        echo "Deleted $name (will be recreated by ReplicaSet)\n";
    }
}

forceRestartPods($client, ['app' => 'my-app', 'env' => 'staging']);
```

Retry with Exponential Backoff

```php
function withRetry(callable $fn, int $attempts = 3, float $delay = 1.0): mixed
{
    for ($i = 1; $i <= $attempts; $i++) {
        try {
            return $fn();
        } catch (\GuzzleHttp\Exception\ServerException $e) {
            if ($i === $attempts) throw $e;
            sleep((int) ($delay * 2 ** ($i - 1)));
        } catch (\GuzzleHttp\Exception\ClientException $e) {
            if ($e->getResponse()->getStatusCode() === 429) {
                sleep((int) ($e->getResponse()->getHeader('Retry-After')[0] ??
5));
                continue;
            }
            throw $e;
        }
    }
}

$pods = withRetry(fn() => $client->pods()->find());
```

Run a One-off Job and Collect Output

```php
function runJobAndGetLogs(Client $client, string $name, array $command, string
$image): string
{
    $jobSpec = [
        'metadata' => ['name' => $name],
```

```
            'spec'     => [
                'backoffLimit' => 0,
                'ttlSecondsAfterFinished' => 600,
                'template' => ['spec' => [
                    'restartPolicy' => 'Never',
                    'containers'    => [['name' => 'runner', 'image' => $image,
'command' => $command]],
                ]],
            ],
        ];

        $client->jobs()->create(new Job($jobSpec));
        $result = waitForJob($client, $name, 120);

        $pods = $client->pods()->setLabelSelector(['job-name' => $name])->find();
        foreach ($pods as $pod) {
            return $client->pods()->logs($pod->getMetadata('name'));
        }

        return '';
    }

    $output = runJobAndGetLogs($client, 'php-task-' . time(), ['php', '-r', 'echo
phpversion();'], 'php:8.2');
    echo "Output: $output\n";
```

Suspend / Resume All CronJobs

```php
function setCronJobsSuspended(Client $client, bool $suspend): void
{
    foreach ($client->cronJobs()->find() as $cj) {
        $client->cronJobs()->patch($cj, ['spec' => ['suspend' => $suspend]]);
        echo ($suspend ? 'Suspended' : 'Resumed') . ': ' . $cj-
>getMetadata('name') . "\n";
    }
}

setCronJobsSuspended($client, true);   // pause all before a deploy
setCronJobsSuspended($client, false);  // resume after
```

Watch Pod Restarts (Alert on CrashLoop)

```php
function alertOnCrashLoop(Client $client, callable $alertFn): void
{
    $seen = [];

    $client->pods()->watch(function ($type, $pod) use (&$seen, $alertFn) {
        $name     = $pod->getMetadata('name');
        $statuses = $pod->getStatus()['containerStatuses'] ?? [];

        foreach ($statuses as $cs) {
            $key     = "$name/" . $cs['name'];
            $reason  = $cs['state']['waiting']['reason'] ?? null;
            $count   = $cs['restartCount'] ?? 0;

            if ($reason === 'CrashLoopBackOff' && !in_array($key, $seen)) {
                $alertFn("CrashLoopBackOff: Pod=$name restarts=$count");
                $seen[] = $key;
            }
        }
        return true;
    });
}

alertOnCrashLoop($client, function ($msg) {
    // Send to Slack, PagerDuty, email, etc.
    error_log("[ALERT] $msg");
});
```

k8s-client-php — kubectl vs PHP Client Cheatsheet

Side-by-side reference: what you'd type in kubectl vs the PHP equivalent.

Pods

kubectl	PHP
`kubectl get pods`	`$client->pods()->find()`

kubectl	PHP
`kubectl get pods -n staging`	`$client->setNamespace('staging'); $client->pods()->find()`
`kubectl get pods -A`	`$client->setNamespace(''); $client->pods()->find()`
`kubectl get pod my-pod`	`$client->pods()->findByName('my-pod')`
`kubectl get pods -l app=nginx`	`$client->pods()->setLabelSelector(['app'=>'nginx'])->find()`
`kubectl describe pod my-pod`	`$pod = $client->pods()->findByName('my-pod'); $pod->getStatus(); $pod->getSpec()`
`kubectl logs my-pod`	`$client->pods()->logs('my-pod')`
`kubectl logs my-pod --tail=100`	`$client->pods()->logs('my-pod', ['tailLines'=>100])`
`kubectl logs my-pod -p`	`$client->pods()->logs('my-pod', ['previous'=>true])`
`kubectl logs my-pod -c sidecar`	`$client->pods()->logs('my-pod', ['container'=>'sidecar'])`
`kubectl delete pod my-pod`	`$client->pods()->deleteByName('my-pod')`
`kubectl delete pod my-pod --grace-period=0`	`$client->pods()->delete($pod, ['gracePeriodSeconds'=>0])`

Deployments

kubectl	PHP
`kubectl get deployments`	`$client->deployments()->find()`

kubectl	PHP
`kubectl get deploy my-app`	`$client->deployments()->findByName('my-app')`
`kubectl scale deploy my-app --replicas=5`	`$client->deployments()->patch($dep, ['spec'=>['replicas'=>5]])`
`kubectl set image deploy/my-app app=myapp:2.0`	`$client->deployments()->patch($dep, ['spec'=>['template'=>['spec'=>['containers'=>[['name'=>'app','image'=>'myapp:2.0']]]]]])`
`kubectl rollout status deploy/my-app`	`waitForDeployment($client, 'my-app')`
`kubectl delete deploy my-app`	`$client->deployments()->deleteByName('my-app')`
`kubectl apply -f deploy.yaml`	`$client->deployments()->create($model)` or `update($model)`

Services

kubectl	PHP
`kubectl get services`	`$client->services()->find()`
`kubectl get svc my-svc`	`$client->services()->findByName('my-svc')`
`kubectl describe svc my-svc`	`$svc->getSpec(); $svc->getStatus()`
`kubectl delete svc my-svc`	`$client->services()->deleteByName('my-svc')`

ConfigMaps & Secrets

kubectl	PHP
`kubectl get configmaps`	`$client->configMaps()->find()`
`kubectl get cm app-config -o json`	`$client->configMaps()->findByName('app-config')`
`kubectl create configmap app-config --from-literal=KEY=val`	`$client->configMaps()->create(new ConfigMap([...'data'=>['KEY'=>'val']...]))`
`kubectl delete cm app-config`	`$client->configMaps()->deleteByName('app-config')`
`kubectl get secrets`	`$client->secrets()->find()`
`kubectl get secret db-creds -o jsonpath='{.data.password}'`	`base64_decode($secret->getAttribute('data')['password'])`
`kubectl create secret generic db-creds --from-literal=password=x`	`$client->secrets()->create(new Secret([...'data'=>['password'=>base64_encode('x')]...]))`
`kubectl delete secret db-creds`	`$client->secrets()->deleteByName('db-creds')`

Namespaces

kubectl	PHP
`kubectl get namespaces`	`$client->namespaces()->find()`
`kubectl create namespace team-alpha`	`$client->namespaces()->create(new NamespaceModel(['metadata'=>['name'=>'team-alpha']]))`
`kubectl delete namespace team-alpha`	`$client->namespaces()->deleteByName('team-alpha')`

Storage

kubectl	PHP
`kubectl get pvc`	`$client->persistentVolumeClaims()->find()`
`kubectl get pv`	`$client->persistentVolumes()->find()`
`kubectl get storageclass`	`$client->storageClasses()->find()`
`kubectl delete pvc my-pvc`	`$client->persistentVolumeClaims()->deleteByName('my-pvc')`

Jobs & CronJobs

kubectl	PHP
`kubectl get jobs`	`$client->jobs()->find()`
`kubectl get job my-job`	`$client->jobs()->findByName('my-job')`
`kubectl delete job my-job`	`$client->jobs()->deleteByName('my-job', ['propagationPolicy'=>'Foreground'])`
`kubectl get cronjobs`	`$client->cronJobs()->find()`
`kubectl patch cronjob my-cj -p '{"spec":{"suspend":true}}'`	`$client->cronJobs()->patch($cj, ['spec'=>['suspend'=>true]])`

StatefulSets & DaemonSets

kubectl	PHP
`kubectl get statefulsets`	`$client->statefulSets()->find()`
`kubectl scale sts mysql --replicas=5`	`$client->statefulSets()->patch($sts, ['spec'=>['replicas'=>5]])`
`kubectl get daemonsets`	`$client->daemonSets()->find()`

Ingress

kubectl	PHP
`kubectl get ingress`	`$client->ingresses()->find()`
`kubectl get ing my-ingress`	`$client->ingresses()->findByName('my-ingress')`
`kubectl delete ingress my-ingress`	`$client->ingresses()->deleteByName('my-ingress')`

RBAC

kubectl	PHP
`kubectl get roles`	`$client->roles()->find()`
`kubectl get rolebindings`	`$client->roleBindings()->find()`
`kubectl get clusterroles`	`$client->clusterRoles()->find()`
`kubectl get clusterrolebindings`	`$client->clusterRoleBindings()->find()`

kubectl	PHP
`kubectl get serviceaccounts`	`$client->serviceAccounts()->find()`
`kubectl create serviceaccount ci-sa`	`$client->serviceAccounts()->create(new ServiceAccount(['metadata'=>['name'=>'ci-sa']]))`

Nodes

kubectl	PHP
`kubectl get nodes`	`$client->nodes()->find()`
`kubectl get node node-1`	`$client->nodes()->findByName('node-1')`
`kubectl describe node node-1`	`$node->getStatus()`

Events

kubectl	PHP
`kubectl get events`	`$client->events()->find()`
`kubectl get events --field-selector type=Warning`	`$client->events()->setFieldSelector(['type'=>'Warning'])->find()`
`kubectl get events -w`	`$client->events()->watch(function($t, $e) { return true; })`

Watch

kubectl	PHP
`kubectl get pods -w`	`$client->pods()->watch(function($type, $pod){ return true; })`
`kubectl get deployments -w`	`$client->deployments()->watch(function($type, $dep){ return true; })`
`kubectl get events -w`	`$client->events()->watch(function($type, $evt){ return true; })`

Label Selectors

kubectl	PHP
`kubectl get pods -l app=nginx`	`$client->pods()->setLabelSelector(['app'=>'nginx'])->find()`
`kubectl get pods -l app=nginx,env=prod`	`$client->pods()->setLabelSelector(['app'=>'nginx','env'=>'prod'])->find()`
`kubectl get pods --field-selector status.phase=Running`	`$client->pods()->setFieldSelector(['status.phase'=>'Running'])->find()`

Cluster Info

kubectl	PHP
`kubectl version`	`$client->getVersion()`
`kubectl cluster-info`	Uses the `master` URL configured in Client

kubectl	PHP
`kubectl api-resources`	`$client->customResourceDefinitions()->find()`

HTTP Status Code Reference

Code	kubectl equivalent	Meaning
`200`	Success	Resource returned/updated
`201`	Created	Resource created
`401`	error: Unauthorized	Bad/expired token
`403`	Error from server (Forbidden)	RBAC missing
`404`	Error from server (NotFound)	Resource doesn't exist
`409`	Error from server (Conflict)	Resource already exists
`422`	error: Invalid	Spec validation failed
`429`	—	Rate limited — add retry
`500`	—	K8s API bug

k8s-client-php — Auth & RBAC Cheatsheet

Auth Methods at a Glance

```
Environment        Recommended Auth Method
──────────────────────────────────────────────────────
Local dev          kubeconfig  (Client::fromKubeConfig())
CI/CD pipeline     ServiceAccount token  (kubectl create token)
In-cluster app     Mounted SA token  (/var/run/secrets/...)
```

```
Cross-account AWS  IRSA + EKS token  (aws eks get-token)
GKE Workload ID    Workload Identity + SA token
```

---

kubeconfig Auth

```php
// Default: uses ~/.kube/config, current-context
$client = Client::fromKubeConfig();

// Specific context
$client = Client::fromKubeConfig(context: 'prod-cluster');

// Specific kubeconfig file
$client = Client::fromKubeConfig(kubeConfigPath: '/tmp/custom.yaml');

// Get context names
// $ kubectl config get-contexts
// $ kubectl config current-context
```

In-Cluster (Pod) Auth

```php
// Kubernetes mounts these automatically into every Pod:
$token =
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/token');
$ca    = '/var/run/secrets/kubernetes.io/serviceaccount/ca.crt';
$ns    =
file_get_contents('/var/run/secrets/kubernetes.io/serviceaccount/namespace');

$client = new Client([
    'master'    => 'https://kubernetes.default.svc',
    'ca_cert'   => $ca,
    'token'     => $token,
    'namespace' => $ns,
]);
```

Pod must have a ServiceAccount with the right RBAC permissions.

ServiceAccount Token (External)

```
# K8s 1.24+ — generate a short-lived token
kubectl create token my-sa -n default --duration=1h

# Or create a long-lived Secret-based token (older)
kubectl apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: my-sa-token
  annotations:
    kubernetes.io/service-account.name: my-sa
type: kubernetes.io/service-account-token
EOF
kubectl get secret my-sa-token -o jsonpath='{.data.token}' | base64 -d
```

```php
$client = new Client([
    'master'  => 'https://k8s.example.com:6443',
    'ca_cert' => '/path/to/ca.crt',
    'token'   => 'eyJhbGci...',
]);
```

RBAC — Common Roles

Read-only access to pods and logs

```php
new Role([
    'metadata' => ['name' => 'pod-reader', 'namespace' => 'default'],
    'rules' => [[
        'apiGroups' => [''],
        'resources' => ['pods', 'pods/log', 'pods/status'],
        'verbs'     => ['get', 'list', 'watch'],
    ]],
])
```

Deployment manager (create/update, no delete)

```php
new Role([
    'metadata' => ['name' => 'deployer', 'namespace' => 'production'],
    'rules' => [
        ['apiGroups' => ['apps'], 'resources' => ['deployments'], 'verbs' =>
['get','list','watch','create','update','patch']],
        ['apiGroups' => [''],     'resources' => ['configmaps','secrets'],
'verbs' => ['get','list']],
    ],
])
```

Full namespace admin

```php
new Role([
    'metadata' => ['name' => 'ns-admin', 'namespace' => 'team-alpha'],
    'rules' => [[
        'apiGroups' => ['', 'apps', 'batch', 'networking.k8s.io'],
        'resources' => ['*'],
        'verbs'     => ['*'],
    ]],
])
```

Cluster-wide read-only (ClusterRole)

```php
new ClusterRole([
    'metadata' => ['name' => 'cluster-viewer'],
    'rules' => [
        ['apiGroups' => ['', 'apps', 'batch'], 'resources' => ['*'], 'verbs' =>
['get', 'list', 'watch']],
        ['apiGroups' => [''],                  'resources' => ['nodes',
'namespaces', 'persistentvolumes'], 'verbs' => ['get', 'list']],
    ],
])
```

RBAC — Common apiGroups + Resources

Resource	apiGroup	Resource name
Pods, Services, ConfigMaps, Secrets	`""` (core)	`pods` , `services` , `configmaps` , `secrets`
Pod logs	`""`	`pods/log`
Pod exec	`""`	`pods/exec`
Deployments, ReplicaSets	`apps`	`deployments` , `replicasets`
StatefulSets, DaemonSets	`apps`	`statefulsets` , `daemonsets`
Jobs, CronJobs	`batch`	`jobs` , `cronjobs`
Ingresses, NetworkPolicies	`networking.k8s.io`	`ingresses` , `networkpolicies`
Roles, RoleBindings	`rbac.authorization.k8s.io`	`roles` , `rolebindings`
ClusterRoles, ClusterRoleBindings	`rbac.authorization.k8s.io`	`clusterroles` , `clusterrolebindings`
PVs, PVCs	`""` / `storage.k8s.io`	`persistentvolumes` , `persistentvolumeclaims`
StorageClasses	`storage.k8s.io`	`storageclasses`
Nodes	`""`	`nodes`
Namespaces	`""`	`namespaces`
CRDs	`apiextensions.k8s.io`	`customresourcedefinitions`

RBAC — All Verbs

Verb	HTTP	Description
`get`	GET (single)	Read one resource

Verb	HTTP	Description
`list`	GET (collection)	List resources
`watch`	GET + `?watch=true`	Stream updates
`create`	POST	Create resource
`update`	PUT	Replace resource
`patch`	PATCH	Partial update
`delete`	DELETE	Delete resource
`deletecollection`	DELETE (collection)	Delete many
`*`	All	All verbs

Bind SA to Role — Full Example

```php
// 1. ServiceAccount
$client->serviceAccounts()->create(new ServiceAccount([
    'metadata' => ['name' => 'ci-sa', 'namespace' => 'default'],
]));

// 2. Role
$client->roles()->create(new Role([
    'metadata' => ['name' => 'ci-role', 'namespace' => 'default'],
    'rules'    => [
        ['apiGroups' => ['apps'], 'resources' => ['deployments'], 'verbs' =>
['get','patch','update']],
        ['apiGroups' => [''],     'resources' => ['configmaps'],  'verbs' =>
['get','create','update']],
    ],
]));

// 3. RoleBinding
$client->roleBindings()->create(new RoleBinding([
    'metadata' => ['name' => 'ci-binding', 'namespace' => 'default'],
    'subjects' => [['kind' => 'ServiceAccount', 'name' => 'ci-sa', 'namespace'
=> 'default']],
```

```
    'roleRef'  => ['apiGroup' => 'rbac.authorization.k8s.io', 'kind' => 'Role',
'name' => 'ci-role'],
]));

// 4. Get token for this SA
// $ kubectl create token ci-sa -n default
```

Bind SA to ClusterRole (cluster-wide)

```php
$client->clusterRoleBindings()->create(new ClusterRoleBinding([
    'metadata' => ['name' => 'dashboard-viewer'],
    'subjects' => [['kind' => 'ServiceAccount', 'name' => 'dashboard-sa',
'namespace' => 'kube-system']],
    'roleRef'  => ['apiGroup' => 'rbac.authorization.k8s.io', 'kind' =>
'ClusterRole', 'name' => 'cluster-viewer'],
]));
```

Test RBAC Permissions

```
# Can this SA list pods?
kubectl auth can-i list pods --as=system:serviceaccount:default:ci-sa

# Can it create deployments?
kubectl auth can-i create deployments --as=system:serviceaccount:default:ci-sa
-n production

# List all permissions for a SA
kubectl auth can-i --list --as=system:serviceaccount:default:ci-sa
```

RBAC Troubleshooting

Symptom	Cause	Fix
`403 Forbidden` on pods	SA has no Role	Create Role + RoleBinding
`403 Forbidden` on nodes	Needs ClusterRole	Use ClusterRole + ClusterRoleBinding
`403 Forbidden` on another namespace	Role is namespaced	Create Role+RoleBinding in that namespace
Token expired	Short-lived token	`kubectl create token` again or use Secret-based token
`401 Unauthorized`	Wrong/missing token	Verify token with `kubectl auth whoami`

k8s-client-php — CronJob Schedule Cheatsheet

Cron Syntax

```
┌──────────── minute        (0–59)
│ ┌────────── hour          (0–23)
│ │ ┌──────── day of month  (1–31)
│ │ │ ┌────── month         (1–12 or JAN–DEC)
│ │ │ │ ┌──── day of week   (0–6 or SUN–SAT, 0=Sunday)
│ │ │ │ │
* * * * *
```

Common Schedules

Schedule	Meaning	Use Case
`* * * * *`	Every minute	Testing only
`*/5 * * * *`	Every 5 minutes	Health checks, polling

Schedule	Meaning	Use Case
`*/15 * * * *`	Every 15 minutes	Metrics collection
`0 * * * *`	Every hour (on the hour)	Hourly reports
`0 */2 * * *`	Every 2 hours	Cache warming
`0 6 * * *`	Every day at 6am UTC	Daily jobs
`0 6 * * 1-5`	Mon–Fri at 6am	Business hours jobs
`0 0 * * 0`	Every Sunday midnight	Weekly cleanup
`0 0 1 * *`	1st of every month	Monthly billing
`0 0 1 1 *`	1st Jan every year	Annual archival
`30 3 * * *`	Every day at 3:30am	Backups (off-peak)
`0 9,17 * * 1-5`	9am & 5pm weekdays	Twice-daily sync

PHP CronJob Creation

php

```php
use Maclof\Kubernetes\Client;
use Maclof\Kubernetes\Models\CronJob;

$client = Client::fromKubeConfig();

function createCronJob(Client $client, string $name, string $schedule, array
$command, string $image): void
{
    $cronJob = new CronJob([
        'metadata' => ['name' => $name, 'namespace' => 'default'],
        'spec' => [
            'schedule'                  => $schedule,
            'concurrencyPolicy'         => 'Forbid',
            'successfulJobsHistoryLimit' => 3,
            'failedJobsHistoryLimit'    => 1,
            'startingDeadlineSeconds'   => 300,
            'jobTemplate' => ['spec' => [
```

```
                'backoffLimit'            => 2,
                'ttlSecondsAfterFinished' => 86400,
                'template' => ['spec' => [
                    'restartPolicy' => 'OnFailure',
                    'containers'    => [['name' => 'job', 'image' => $image,
'command' => $command]],
                ]],
            ]],
        ],
    ]);
    $client->cronJobs()->create($cronJob);
    echo "CronJob '$name' created: $schedule\n";
}

// Examples
createCronJob($client, 'daily-report',   '0 6 * * *',     ['php', 'artisan',
'report:daily'],   'myapp:latest');
createCronJob($client, 'hourly-sync',    '0 * * * *',     ['php', 'artisan',
'sync:data'],      'myapp:latest');
createCronJob($client, 'weekly-cleanup', '0 0 * * 0',     ['php', 'artisan',
'cleanup:old'],    'myapp:latest');
createCronJob($client, 'every-5min',     '*/5 * * * *',   ['php', 'artisan',
'queue:heartbeat'], 'myapp:latest');
```

ConcurrencyPolicy Options

```php
'concurrencyPolicy' => 'Allow',    // Run overlapping jobs (default)
'concurrencyPolicy' => 'Forbid',   // Skip if previous still running
'concurrencyPolicy' => 'Replace',  // Kill previous, start new
```

Manage CronJobs

```php
// List all
$client->cronJobs()->find()

// Get one
$cj = $client->cronJobs()->findByName('daily-report')
```

```php
// Check schedule
$cj->getSpec()['schedule']
$cj->getStatus()['lastScheduleTime']
$cj->getSpec()['suspend']

// Suspend (pause)
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => true]])

// Resume
$client->cronJobs()->patch($cj, ['spec' => ['suspend' => false]])

// Change schedule
$client->cronJobs()->patch($cj, ['spec' => ['schedule' => '0 8 * * *']])

// Delete
$client->cronJobs()->deleteByName('daily-report')

// Suspend all
foreach ($client->cronJobs()->find() as $cj) {
    $client->cronJobs()->patch($cj, ['spec' => ['suspend' => true]]);
}
```

Manually Trigger a CronJob

```php
function triggerCronJob(Client $client, string $cronJobName): string
{
    $cj      = $client->cronJobs()->findByName($cronJobName);
    $jobName = $cronJobName . '-manual-' . time();

    $job = new \Maclof\Kubernetes\Models\Job([
        'metadata' => ['name' => $jobName, 'namespace' => 'default'],
        'spec'     => $cj->getSpec()['jobTemplate']['spec'],
    ]);

    $client->jobs()->create($job);
    echo "Triggered: $jobName\n";
    return $jobName;
}
```

```php
$jobName = triggerCronJob($client, 'daily-report');
// Then monitor: waitForJob($client, $jobName)
```

CronJob vs Job Quick Pick

```
Need to run something...
  ├── Once, right now              → Job
  ├── Once, at a specific time     → CronJob (one-time schedule) or Job with
delay
  ├── Every N minutes              → CronJob  */N * * * *
  ├── Daily                        → CronJob  0 H * * *
  ├── Weekly                       → CronJob  0 H * * DOW
  └── Monthly                      → CronJob  0 H 1 * *
```

Timezone Reference

```php
'timezone' => 'UTC',                    // Default
'timezone' => 'America/New_York',       // EST/EDT
'timezone' => 'America/Los_Angeles',    // PST/PDT
'timezone' => 'Europe/London',          // GMT/BST
'timezone' => 'Europe/Paris',           // CET/CEST
'timezone' => 'Asia/Tokyo',             // JST
'timezone' => 'Asia/Kolkata',           // IST
'timezone' => 'Australia/Sydney',       // AEST/AEDT
```

> **Requires** Kubernetes 1.27+ and `CronJobTimeZone` feature gate enabled.

List Jobs Created by a CronJob

```php
// Jobs created by a CronJob have a label: job-name={cronjob-name}-{timestamp}
// Or find by ownerReferences

$jobs = $client->jobs()->find();
```

```
  foreach ($jobs as $job) {
      $owners = $job->getMetadata()['ownerReferences'] ?? [];
      foreach ($owners as $owner) {
          if ($owner['kind'] === 'CronJob' && $owner['name'] === 'daily-report')
  {
              $name      = $job->getMetadata('name');
              $startTime = $job->getStatus()['startTime'] ?? '-';
              $succeeded = $job->getStatus()['succeeded'] ?? 0;
              echo "$name  started=$startTime  succeeded=$succeeded\n";
          }
      }
  }
```